Learning Practical Policies with Upside-Down Reinforcement Learning

Master's Thesis submitted to the
Faculty of Informatics of the *Università della Svizzera Italiana*
in partial fulfillment of the requirements for the degree of
Master of Science in Artificial Intelligence

presented by
Jacopo Di Ventura

under the supervision of
Dr. Jürgen Schmidhuber
co-supervised by
Dylan R. Ashley, Dr. Francesco Faccio, Vincent Herrmann

September 2024

I certify that except where due acknowledgement has been given, the work presented in this thesis is that of the author alone; the work has not been submitted previously, in whole or in part, to qualify for any other academic award; and the content of the thesis is the result of work which has been carried out since the official commencement date of the approved research program.

Jacopo Di Ventura
Lugano, 9 September 2024

To my beloved

Someone said ...

Someone

Abstract

This thesis presents Upside Down Reinforcement Learning with Policy Generators (UDRLPG), a novel reinforcement learning (RL) approach designed to learn a generator of deep neural network policies conditioned on return commands. Goal-conditioned RL aims at learning optimal policies, given goals encoded as command inputs. Here, we abstract this concept by employing goal-conditioned neural networks in the form of Hypernetworks—a kind of Fast Weight Programmers (FWP)— which learn to decode input commands representing a desired expected return into context-specific weight matrices. A single network generates the target network's hidden layers in smaller slices conditioned on an embedding and the command, while other networks handle input and output weights. The slices are then concatenated to obtain the weight matrix. By exploiting exploration in parameter space, our method is able to produce increasingly higher-reward policies. The proposed approach streamlines comparable techniques by removing the need for an evaluator to update the weights of the generator. The removal of the evaluator translates primarily into higher variance in last returns. The fragility introduced by this simplification is mitigated by decoupling the sampling probability in the replay buffer from the absolute number of policies in it, which, together with a fine-tuned weighting strategy and hyperparameters, improves the empirical convergence of the algorithm. Compared with existing algorithms, UDRLPG achieves competitive performance and high returns, sometimes outperforming more complex architectures. Our experiments show how a trained generator can create policies that achieve different returns on the reward spectrum in a zero-shot manner without further training. The proposed method appears to be effective in mitigating some of the challenges associated with learning highly multimodal functions. Future work should explore additional techniques to stabilize training, leading to lower variance and faster convergence during training.

Acknowledgements

Contents

Contents xi

List of Figures xiii

List of Tables xv

1 Introduction 1
1.1 Overview . . . 2

2 Background 3
2.1 Supervised Learning . . . 3
2.2 Artificial Neural Networks . . . 3
2.2.1 Activation Functions . . . 3
2.2.2 Forward Pass . . . 4
2.2.3 Backpropagation . . . 4
2.3 Hypernetworks . . . 5
2.4 Reinforcement Learning . . . 5
2.4.1 Problem Definition . . . 6
2.4.2 Q-Learning . . . 6
2.4.3 SARSA . . . 7
2.4.4 Deep Reinforcement Learning . . . 7
2.4.5 Policy Optimization . . . 7
2.4.6 Actor-Critic . . . 8
2.4.7 Goal-conditioning . . . 8
2.4.8 Upside-down Reinforcement Learning . . . 8
2.4.9 Deep Q-Network . . . 9
2.4.10 Parameter-based Methods . . . 9
2.4.11 GoGePo . . . 9
2.4.12 Policy Fingerprinting . . . 10
2.5 Autoencoders . . . 10
2.5.1 Standard Autoencoders . . . 10
2.5.2 Variational Autoencoders . . . 10

3 Related Works 13
3.1 Fast Weight Programmers . . . 13
3.2 Parameter-based Value Functions . . . 13
3.3 GoGePo . . . 14
3.4 Evolution Strategies . . . 14
3.5 Policy Gradients with Parameter-Based Exploration . . . 14
3.6 Upside Down Reinforcement Learning . . . 14

4 Method **15**
4.1 Motivation . . . 15
4.2 UDRLPG . . . 16
4.2.1 Exploration and expoitation . . . 16
4.2.2 Replay buffer and buckets . . . 18
4.3 Observation normalization . . . 19
4.4 Hypernetworks . . . 20
4.5 Differences with GoGePo . . . 21

5 Experimental Setup **23**
5.1 Environments . . . 23
5.1.1 InvertedPendulum-v4 . . . 23
5.1.2 Swimmer-v4 . . . 24
5.1.3 Hopper-v4 . . . 24
5.2 Evaluation . . . 25
5.2.1 Evaluation . . . 25
5.2.2 Identity Evaluation . . . 25
5.3 Hyperparameter tuning . . . 26
5.4 Comparison Algorithms . . . 26

6 Results **31**
6.1 InvertedPendulum-v4 . . . 31
6.2 Swimmer-v4 . . . 33
6.3 Hopper-v4 . . . 35

7 Discussion **37**

8 Conclusion **43**
8.1 Future Work . . . 43

Glossary **45**

Bibliography **47**

Figures

4.1 Visualization of update step in UDRLPG 17
4.2 Visualization of dynamic buckets 20

5.1 Illustration of InvertedPendulum-v4. Source: Gymnasium documentation 23
5.2 Illustration of Swimmer-v4. Source: Gymnasium documentation 24
5.3 Illustration of Hopper-v4. Source: Gymnasium documentation 25

6.1 Performance of policies obtained with UDRLPG, GoGePo, and DDPG during training in InvertedPendulum-v4. Curves show the mean return and 95% bootstrapped confidence intervals from 20 independent runs as a function of environment interactions. All algorithms converge to comparable performance. 31
6.2 Variance of last returns obtained with UDRLPG, GoGePo, and DDPG in InvertedPendulum-v4: Mean return and 95% bootstrapped confidence intervals from 20 runs at final evaluation, as a function of environment interactions. GoGePo shows lower variance compared to the other methods. 32
6.3 Mean returns over 10 episodes of policies from the generator as a function of the given command in InvertedPendulum-v4. Results are from five independent runs. The method can produce increasingly higher return policies. 32
6.4 Performance of policies obtained with UDRLPG, GoGePo, and DDPG during training in Swimmer-v4. Curves show the mean return and 95% bootstrapped confidence intervals from 20 independent runs as a function of environment interactions. UDRLPG reaches a slightly lower return than GoGePo, while outperforming DDDPG. 33
6.5 Variance of last returns obtained with UDRLPG, GoGePo, and DDPG in Swimmer-v4: Mean return and 95% bootstrapped confidence intervals from 20 runs at final evaluation, as a function of environment interactions. GoGePo shows lower variance compared to the other methods. 33
6.6 Mean returns over 10 episodes of policies from the generator as a function of the given command in Swimmer-v4. Results are from five independent runs. The method can produce increasingly higher return policies. 34
6.7 Performance of policies obtained with UDRLPG, GoGePo, and DDPG during training in Hopper-v4. Curves show the mean return and 95% bootstrapped confidence intervals from 20 independent runs as a function of environment interactions. UDRLPG and GoGePo converge to comparable performance. 35
6.8 Variance of last returns obtained with UDRLPG, GoGePo, and DDPG in Hopper-v4: Mean return and 95% bootstrapped confidence intervals from 20 runs at final evaluation, as a function of environment interactions. Both GoGePo and UDRLPG show high variance. UDRLPG's last returns are distributed over the spectrum. GoGePo's last returns are concentrated into two clusters 35
6.9 Mean returns over 10 episodes of policies from the generator as a function of the given command in Hopper-v4. Results are from five independent runs. The method can produce increasingly higher return policies. 36

7.1 Performance of policies obtained with UDRLPG during training in InvertedPendulum-v4 (a), Swimmer-v4 (b), and Hopper-v4 (c), using three different strategies for the replay buffer: performance buckets with weighted sampling (the proposed approach); no performance buckets with uniform sampling; no performance buckets with weighted sampling. Curves show the mean return and 95% bootstrapped confidence intervals from 20 independent runs as a function of environment interactions. Weighting the buffer without buckets outperforms a uniform strategy. The proposed strategy outperforms all other strategies in every environment. 38
7.2 Performance of policies obtained with UDRLPG during training in InvertedPendulum-v4 (a), Swimmer-v4 (b), and Hopper-v4 (c), using either uniform or weighted sampling from the replay buffer. Curves show the mean return and 95% bootstrapped confidence intervals from 20 independent runs as a function of environment interactions. The proposed weighting strategy outperforms uniform weighting in all environments. 39
7.3 t-SNE applied *indepentently* to each buffer of five independent runs. Color gradient represents the return of each policy. Buffer dumped at the end of training. Each buffer contains 2999 policies. Environment: Swimmer-v4 41
7.4 t-SNE applied to policies in the buffers of five independent runs. Buffers dumped at the end of training and merged together. Environment: Swimmer-v4 42

Tables

5.1 Hyperparameters for UDRLPG 27
5.2 Hyperparameters for GoGePo 28
5.3 Hyperparameters for DDPG 29

6.1 Statistics for the last return (last evaluation) of 20 independent runs. Mean return is the mean of the last returns; std is the standard deviation of the last returns; max is the maximum last return; min is the minimum last reward (20 evaluations) for UDRLPG, GoGePo, and DDPG in InvertedPendulum-v4. 32
6.2 Statistics for the last return (last evaluation) of 20 independent runs. Mean return is the mean of the last returns; std is the standard deviation of the last returns; max is the maximum last return; min is the minimum last reward (20 evaluations) for UDRLPG, GoGePo, and DDPG in Swimmer-v4. 34
6.3 Statistics for the last return (last evaluation) of 20 independent runs. Mean return is the mean of the last returns; std is the standard deviation of the last returns; max is the maximum last return; min is the minimum last reward (20 evaluations) for UDRLPG, GoGePo, and DDPG in Hopper-v4. 36

Chapter 1

Introduction

Artificial Intelligence (AI) is a field of study focused on enabling machines to execute tasks that are typically straightforward for humans yet historically challenging for computers. To many, the concept of AI may appear rather new. In reality, the idea of mechanizing human thought dates back millennia (McCorduck and Cfe [2004]; Buchanan [2005]; Sharkey [2007a,b]). Significant contributions to the field of AI considerably predate the coinage of the term (McCarthy et al. [2006]). Examples include the chain rule in 1676 (Leibniz [2012]) and linear neural networks from circa 1800 (Stigler [1981]).

The significance of AI, especially in modern society, cannot be overstated. AI's impact is profound and far-reaching. Technologies taking advantage of its capabilities have permeated practically every field, transforming how industries operate, and people interact with technology. Notable examples include the groundbreaking advancements achieved by AlphaFold (Jumper et al. [2021]), which revolutionized the field of biology by accurately predicting protein structures. Another significant advancement has been achieved in the field of self-driving vehicles, as they are now capable of autonomously navigating complex road systems with increased precision and safety, owing to sophisticated deep learning algorithms (seed Goodall [2014]).

Within the broad domain of AI, reinforcement learning (RL) stands out as a particularly challenging but important area of research. RL is a paradigm where an agent learns to interact with an environment through actions. The agent receives positive or negative reinforcement in the form of rewards, which allow it to gradually improve its policy: the strategy that dictates its actions. RL is remarkable for its ability to address problems associated with sequential decision-making, particularly in scenarios where the consequences of actions are not immediately apparent. Notable examples of success in the field of RL include AlphaGo (Silver et al. [2017]), the first AI to defeat a world champion in the game of Go, and TD-Gammon, which learned to master the video game Backgammon exploring strategies not yet pursued by human players (Tesauro [1994]). More recently, RL has been employed to optimize chip design (Rajarshi [2022]) and to control plasma in fusion reactors (Hiard [2022]), highlighting its ability to solve tasks at the very limit of human ability.

The most notable weakness of RL is sample complexity—the amount of data required for an RL agent to learn an effective policy (Kakade [2003]). Most RL algorithms require many interactions with the environment before reaching an acceptable level of performance. Reducing sample complexity is a key objective for advancing the field, as doing so would allow it to be applicable to a broader range of problems, for instance, where interactions with the environment are computationally or financially expensive.

The method explored in this thesis is Upside Down Reinforcement Learning with Policy Generators (UDRLPG). This thesis will delve into the intricacies of UDRLPG, an RL algorithm designed to generate policies conditioned on a return command. The architecture of the proposed model is built on a Fast Weight Programmer, referred to as the generator, which learns to decode a given command into weights of a policy. The resulting model is capable of generating policies that achieve any arbitrary reward in a zero-shot manner without any additional training. The thesis will explore both the design and theoretical basis of UDRLPG, discuss its potential advantages and disadvantages compared to similar approaches and established RL algorithms, and provide a comprehensive analysis of its underlying mechanisms and components.

1.1 Overview

This thesis is organized into several chapters, each focusing on different aspects of the research on UDRLPG. The following is a summary of the remaining chapters and their contents.

Chapter 2 provides the necessary background for understanding the development of the proposed method, covering topics relevant to the research. Chapter 3 reviews existing literature in the field, including an overview of previous approaches and techniques closely related to UDRLPG, highlighting their similarities and relationships. Chapter 4 details the proposed approach, describing the design and development of the UDRLPG algorithm, including key components, the motivation behind the design choices, and implementation specifics. Chapter 5 outlines the experimental setup used to evaluate the proposed method, including the environments selected for testing and the criteria used to assess its performance. Chapter 6 presents the results of the experiments, providing comprehensive data on the performance of the proposed method across multiple environments. Chapter 7 interprets these results, discusses the implications of the findings, and evaluates the strengths and limitations of the approach, connecting the outcomes to theoretical foundations while offering further analysis of certain aspects of the method. Finally, Chapter 8 summarizes the findings, discusses the overall contributions of the thesis, and offers remarks on the study's impact, suggesting potential directions for future work and areas for further research.

Chapter 2

Background

2.1 Supervised Learning

Supervised learning (SL) is one of the most popular settings in machine learning, in which a model is trained using a dataset containing input values x and their corresponding desired outputs y. The objective is for the model to learn the underlying function that generated the data, ideally ignoring any noise present within the dataset.

In supervised learning, we are given a dataset $D = \{(x_i, y_i)\}_{i=1}^{N}$, where $x_i \in \mathbb{R}^d$ indicates the feature vector of the i-th datapoint and $y_i \in \mathbb{R}$ (for regression tasks) or $y_i \in \{1, 2, \ldots, K\}$ (for classification tasks) is the corresponding label. The goal is to learn a function $f : \mathbb{R}^d \to \mathbb{R}$ or $f : \mathbb{R}^d \to \{1, 2, \ldots, K\}$ that maps inputs to outputs. In the most basic setting, the dataset is typically divided into a training and a testing set. The training set is employed for training the model, while the testing set is used to measure the model's performance on data it has not been exposed to. A critical aspect of supervised learning is balancing overfitting and underfitting. Overfitting occurs when a model learns the training data too well, capturing noise and details that do not generalize to new data and, as a result, making the predictions on unseen data less accurate. For this reason, splitting the dataset into train and test sets is crucial in supervised learning. An overfitted model produces accurate predictions on the training data but performs significantly worse on unseen test data. Overfitting can be seen when the training error L_{train} is low, but the testing error L_{test} is high: $L_{\text{train}} \ll L_{\text{test}}$. Underfitting, on the other hand, occurs when a model is too simple to capture the patterns in the data, resulting in poor performance on both training and testing sets. It is characterized by both high training and testing errors.

To evaluate the performance of supervised learning models, various metrics are used. For regression tasks, the most common is the mean squared error, $\text{MSE} = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2$. For classification tasks, metrics include accuracy, precision, and recall.

2.2 Artificial Neural Networks

Artificial neural networks, often contemporarily referred to as simply neural networks (NNs), are a class of machine learning models. They typically consist of an input layer, one or more hidden layers, and an output layer. Each layer is composed of neurons, which are simple computational units that receive an input, apply a nonlinear transformation, and pass the result to the next layer. The connections between neurons are associated with weights that are adjusted during training to minimize the error, computed with a loss function, in the network predictions.

The operation of a neural network can be divided into two main phases: the forward pass and the backward pass (backpropagation).

2.2.1 Activation Functions

Incorporating nonlinear activation functions is essential in neural networks, as they allow the network to effectively capture and model complex patterns and relationships within the data. In the absence of nonlinearities, a neural network with multiple layers behaves like a single-layer perceptron, as the composition of linear functions results in another linear function. Applying a linear function to another

linear function is equivalent to applying a linear function to the original input. In other words, given two linear functions W_a and W_b it is possible to define W_{ab} such that $W_{ab} = W_b(W_a x)$. Consequently, the NN loses most of its capacity because, in most cases, the output we are trying to predict has a non-linear relationship with the inputs.

The choice of activation function can significantly impact the training dynamics, such as convergence speed. For instance, Rectified Linear Unit (ReLU) (Fukushima [1969]) has risen in popularity because of its ability to mitigate the vanishing gradient problem, a common phenomenon with sigmoid and tanh functions, in which, near the saturated regions, the gradients become very small, leading to slow updates during training and hence slower convergence.

2.2.2 Forward Pass

During the forward pass, the input is passed through the network layer by layer. Each neuron performs a weighted sum of its inputs and applies a nonlinear activation function to this sum to produce its output, which becomes the input of the next layer.

The output $a_j^{(l)}$ of the j-th neuron in layer l is given by:

$$z_j^{(l)} = \sum_i w_{ji}^{(l)} a_i^{(l-1)} + b_j^{(l)}$$

$$a_j^{(l)} = \sigma(z_j^{(l)}).$$

Here, $w_{ji}^{(l)}$ is the weight of the connection from neuron i in layer $l-1$ to neuron j in layer l, $b_j^{(l)}$ is the bias term for neuron j in layer l, and σ is the activation function.

2.2.3 Backpropagation

Backpropagation (Linnainmaa [1970]) is a key algorithm used for training neural networks by optimizing their weights to minimize output error. The error computed with the loss function is propagated backwards through the network from the output layer to the input layer. The loss function L measures the error between the prediction of the network $\hat{y}$ and the true label y.

The process starts with a forward pass through the network. Once the output is computed, the error at the output layer is determined using a loss function.

Backpropagation is used to calculate the gradients of the loss (error) function with respect to each weight and bias in the neural network (NN) by applying the chain rule of calculus iteratively. This is done by propagating the error from the output layer back through the network to all the layers, keeping all intermediate results to avoid redundant calculations. The gradient of the loss with respect to a weight w_{ji} is defined as:

$$\frac{\partial \mathcal{L}}{\partial w_{ji}^{(l)}} = \frac{\partial L}{\partial a_j^{(l)}} \cdot \frac{\partial a_j^{(l)}}{\partial z_j^{(l)}} \cdot \frac{\partial z_j^{(l)}}{\partial w_{ji}^{(l)}}$$

The gradient represents the rate of change of the loss function $\mathcal{L}$ w.r.t. a small change in the weight or bias. Backpropagation provides these gradients for each weight and bias in the network.

Once the gradients are calculated through backpropagation, the weights and biases are optimized to minimize the loss function via gradient descent. Gradient descent is an optimization algorithm that adjusts the weights and biases in the direction that reduces the loss, typically in small steps determined by the learning rate η. For a given weight, the update rule is defined as:

$$w_{ji}^{(l)} \leftarrow w_{ji}^{(l)} - \eta \frac{\partial \mathcal{L}}{\partial w_{ji}^{(l)}}$$

Where, η is a hyperparameter, usually referred to as the learning rate, that controls the size of the step taken toward the direction of the negative gradient.

For completeness, the bias updates rule is defined as:

$$b_j^{(l)} \leftarrow b_j^{(l)} - \eta \frac{\partial \mathcal{L}}{\partial b_j^{(l)}}$$

By iteratively applying these updates, the error is gradually reduced, improving predictions. This iterative process can be stopped once a stopping condition is reached.

2.3 Hypernetworks

The term hypernetwork (HN) (Ha et al. [2016]) refers to a class of FWPs (Schmidhuber [1992, 1993]) designed to output the weights θ of another neural network T (the target network) in the form of smaller slices s_l. A HN is defined as follows

$$H_\phi(c; z_l) = s_l,$$

where c is the context information and z_l is a learned embedding. The learnable parameters of the architecture are $z_l \forall l; \phi$. Hypernetworks have a variety of applications, one of which is the ability to be trained to produce the weights of multiple NNs for solving a task conditioned on some information given as input to the HN.

The forward pass begins by giving as input the context vector c to the HN H, which generates the weights of T, θ. Just like a standard NN, an input x is passed to T, which outputs the prediction $\hat{y}$. The loss can now be computed as $L(y, \hat{y})$. The difference lies in the backward pass, where the error is propagated back to the HN H, and the gradients are computed w.r.t. ϕ and the embeddings z_l, the only learnable parameters in the architecture.

Hypernetworks can dynamically generate weights for the target network, allowing it to adapt to different tasks efficiently. This reduces the need for multiple independent models, saving computational resources and memory. Hypernetworks are beneficial as they allow models to learn new tasks without losing knowledge of previous tasks.

Hypernetworks have been successfully used for zero-shot behavior generalization (Rezaei-Shoshtari et al. [2023]). HNs learn to generalize optimal behavior across a family of tasks by training on the full RL solutions of training tasks composed of optimal policy, rolled-out optimal behavior, and value function parameters. In this approach, HNs are trained to output high-performing policies without any experience in a given related but unseen task by conditioning the generation on task parameters.

2.4 Reinforcement Learning

Reinforcement Learning (RL) (see Sutton [2018] or Kaelbling et al. [1996]) is a powerful machine learning paradigm inspired by behavioral psychology. In RL, an agent learns to make decisions by interacting with its environment. Unlike supervised learning, where the model is trained on a fixed dataset, RL involves a dynamic learning process through trial and error. The agent receives feedback from the environment in the form of rewards, guiding it over time towards optimal behavior.

At the core of theoretical RL is the concept of Markov Decision Process (MDP) (Puterman [2014]; Stratonovich [1960]), which provides a mathematical framework for modeling decision-making problems. An MDP is defined by a set of states S, a set of actions A, a transition function T that describes the probability of moving from one state to another, and a reward function R that assigns rewards to state-action pairs. The objective of the RL agent is to learn a policy π—a mapping from states to actions—that maximizes the expected cumulative reward over time.

RL algorithms can be broadly divided into two categories: model-based and model-free. Model-based RL involves creating a model of the environment's dynamics. This model predicts the next state and reward given a current state and action. The agent uses this model to plan by simulating potential future states and actions, thus allowing it to make informed decisions. Model-based models allow for more sample efficiency because the agent can use the model to simulate experiences. However, building an accurate model of the environment can be difficult, especially in complex or stochastic environments.

Model-free RL, on the other hand, does not rely on a model of the environment. Instead, it focuses on learning a policy or value function directly from interactions with the environment. Model-free methods are generally easier to implement and apply to a wider range of problems compared to model-based methods. Model-free algorithms are further divided into two main categories: Policy gradient algorithms and Q-value-based algorithms.

2.4.1 Problem Definition

A Markov Decision Process (MDP) (Puterman [2014]; Stratonovich [1960]) is defined as $(S, A, P, R, \gamma, \mu_0)$. At each time step t the agent observes a state $s_t \in S$, chooses an action $a_t \in A$ and receives a reward $r_t = R(s_t, a_t)$. The action leads to a new state following the transition probability defined as $\mathcal{P}(s_{t+1}|s_t, a_t)$. Each episode begins in an initial state s_0, selected with probability μ_0. The policy $\pi_\theta : S \to \Delta(A)$ controls the agent, where $\theta \in \Theta$ are the parameters of the policy. A trajectory $\tau \in \mathcal{T}$ is defined as the sequence of state-action pairs that an agent encounters during an episode in the MDP:

$$\tau = (s_{\tau,0}, a_{\tau,0}, s_{\tau,1}, a_{\tau,1}, \ldots, s_{\tau,T}, a_{\tau,T}),$$

where T is the time step at the end of the episode. An episode ends when one of two conditions are met: a time step limit is reached or the encounter of a terminal state (most formalizations set the time step limit as ∞). The return of a trajectory $R(\tau)$ is defined as the (discounted) sum of rewards over the trajectory:

$$R(\tau) = \sum_{t=0}^{T} \gamma^t R(s_{\tau,t}, a_{\tau,t}),$$

where $\gamma \in (0, 1]$ is the discount factor.

The RL problem involves finding the policy π_{θ^*} that maximizes the expected return obtained from the environment:

$$\pi_{\theta^*} = \arg\max_{\pi_\theta} J(\theta),$$

where

$$J(\theta) = \int_T p(\tau|\theta) R(\tau)\, d\tau.$$

Here, $p(\tau|\theta)$ is the distribution over trajectories induced by the policy π with parameters θ in the MDP, given by:

$$p(\tau|\theta) = \mu_0(s_0) \prod_{t=0}^{T} \pi_\theta(a_t|s_t) P(s_{t+1}|s_t, a_t).$$

2.4.2 Q-Learning

Q-learning (Watkins and Dayan [1992]) is an RL algorithm that aims to learn the optimal action-value function, denoted as $Q(s, a)$, which represents the expected cumulative reward of taking action a in state s and following the optimal policy onwards. Q-learning is an off-policy method, meaning that it updates the Q-function based on the maximum expected reward of the next state, independently of the agent's current policy. The update rule for Q-learning is as follows:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right],$$

where α is the learning rate, γ is the discount factor, r is the immediate reward received after transitioning from state s to state s' by taking action a, and $\max_{a'} Q(s', a')$ represents the maximum estimated Q-value for the next state s'. This off-policy nature allows Q-learning to evaluate the optimal policy even when actions are taken according to a different, exploratory behavior policy.

2.4.3 SARSA

State-Action-Reward-State-Action (SARSA) (Sutton [1995]) is an RL algorithm that learns the action-value function $Q(s, a)$. Unlike Q-learning, SARSA updates the Q-function based on the actions dictated by the current policy making it an on-policy algorithm. The update rule incorporates the action a'' taken in the next state s', rather than the greedy action used in Q-learning. The update rule is as follows:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma Q(s', a') - Q(s, a) \right],$$

where a' is the action chosen following the current policy in the next state s'. Because it is on-policy, SARSA is more sensitive to the behavior of the policy, as it directly updates the Q-values based on the actions that the policy would take. Sarsa is on-policy, therefore it will not converge to the optimal Q value as long as exploration occurs. Therefore, exploration must be reduced over time. Given that the policy becomes greedy in the limit, SARSA can converge to optimal Q values, just like Q-learning.

2.4.4 Deep Reinforcement Learning

Deep Reinforcement Learning (Deep RL) is a subfield of machine learning that combines reinforcement learning (RL) with deep learning. In Deep RL, agents learn to make decisions by interacting with an environment with the aim of maximizing cumulative rewards. Deep RL makes use of deep neural networks (DNNs) to approximate complex functions, enabling the agent to handle high-dimensional state and action spaces that traditional RL methods struggle with. In traditional RL, policy and value functions are often represented using tables or linear functions, which limit the scalability and generalization to large, complex problems. With DNNs, we can overcome these limitations by leveraging their ability to approximate highly complex functions, enabling the agent to learn environments that would otherwise be intractable with traditional RL methods.

Deep RL has been successfully applied to various challenging tasks, such as robotics, autonomous driving, and financial trading. Notable examples include TD-Gammon (Tesauro [1994]), and Deep Q-Network (DQN) (Mnih [2013]).

2.4.5 Policy Optimization

Policy optimization algorithms directly parameterize the policy and optimize the parameters to maximize the expected return. The policy specifies the probability of taking action in a state, parameterized by certain variables. Policy Gradient Methods (Sutton et al. [1999]) are a common type of policy optimization algorithm that optimize the policy by following the gradient of the expected reward. REINFORCE (Williams [1992]) is a well-known algorithm in this category, designed to optimize policy parameters directly using gradient ascent. A policy $\pi(\theta)$ maps states s to a probability distribution over actions a, parameterized by θ. The objective is to maximize the expected return $J(\theta)$, defined as $J(\theta) = \mathbb{E}_{\tau \sim \pi(\theta)}[G(\tau)]$, where G_t is the total accumulated reward from time step t onward in an episode.

The policy gradient theorem provides the gradient of the objective function $J(\theta)$ with respect to the policy parameters θ as

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi(\theta)} \left[\sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t|s_t) G_t \right].$$

To approximate this expectation, REINFORCE uses Monte Carlo sampling:

$$\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_{i=1}^{N} \sum_{t=0}^{T} \nabla_\theta \log \pi(a_t^i|s_t^i; \theta) G_t^i$$

$$= \frac{1}{N} \sum_{i=1}^{N} \left[(\sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t|s_t))(\sum_{t=0}^{T} R(s_t^i, a_t^i)) \right],$$

where N is the number of sampled episodes, T is the time step at the end of the episode, and G_t^i is the return observed from the i-th episode starting at time t.

The policy parameters are then updated using gradient ascent:

$$\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta),$$

where α is the learning rate. In practice, this involves initializing the policy parameters θ, generating trajectories by following the policy $\pi(\theta)$, computing returns for each time step in the trajectory, estimating the gradient of the objective using the sampled trajectories, and updating the policy parameters. This process is repeated until a termination condition is met.

Policy Gradient methods have demonstrated significant success across a variety of tasks when applied to Long Short-Term Memory (LSTM) Hochreiter [1997]; Wierstra et al. [2007]. For instance, a policy gradient-trained LSTM served as the core of OpenAI Five, which achieved a remarkable feat by defeating professional human players in the video game Dota 2 (Berner et al. [2019]).

2.4.6 Actor-Critic

Actor-critic (Sutton [1984]) methods integrate both policy-based and value-based techniques. A critic evaluates the policy by learning a value function that is used as a control variate by the actor. These methods offer reduced variance in policy updates compared to pure policy gradient methods. In policy gradient methods, the cumulative reward (return) $G_t = \sum_{t=0}^{T} R(s_t^i, a_t^i)$ is used to update the policy directly, which could lead to high variance. By introducing a value function, the critic provides an estimate of the expected future rewards. In the policy gradient method, the policy π_θ is updated using the gradient of the expected return:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta} \left[\nabla_\theta \log \pi_\theta(a|s) G_t \right],$$

where G_t is the cumulative reward from time step t. In the Actor-Critic method, instead of the cumulative reward G_t, the advantage function $A(s, a)$ is typically used.

$$A_{\pi_\theta}(s_t, a_t) = Q(s_t, a_t) - V_{\pi_\theta}(s_t)$$

where $R(s_t, a_t)$ is the immediate reward, $V_{\pi_\theta}(s_{t+1})$ is the estimated value of the next state, and $V_{\pi_\theta}(s_t)$ is the estimated value of the current state. Using this advantage function, the policy update rule becomes:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta} \left[\nabla_\theta \log \pi_\theta(a|s) A_{\pi_\theta}(s_t, a_t) \right],$$

The critic learns the value function $V_{\pi_\theta}(s)$, it is updated by minimizing the error between its prediction $V_{\pi_\theta}(s_t)$ and the observed return.

2.4.7 Goal-conditioning

Goal-conditioned RL (Schmidhuber [1991]; Schmidhuber and Wahnsiedler [1993]) agents differ from classic RL agents because of their ability to solve more than one task. While the classic RL agent learns to maximize the expected return, goal-conditioned agents learn to solve a number of tasks encoded through a command. UDRL and related approaches employ supervised learning to train goal-conditioned RL agents by receiving command inputs that specify the desired outcome within a certain timeframe. However, in the episodic setting, it is often the case that there is no single sequence of actions or behavior that satisfies a given command. For this reason, a maximum likelihood approach may not be able to capture the variability in the data.

2.4.8 Upside-down Reinforcement Learning

Classic model-free RL algorithms take one of three approaches: they either predict rewards with value functions (Q-learning), maximize rewards using policy gradient methods or combine the two aforementioned approaches (actor-critic). Upside-down reinforcement Learning (UDRL) (Schmidhuber [2019]) is an effort to look at the problem of RL from a different angle, that is, creating a mapping from state and return to action, effectively turning the classical RL formulation upside down. This approach takes the problem of learning to act within an environment closer to a supervised learning task, as the goal of the model now becomes learning a mapping between state, command, and actions rather than learning a mapping from state-action pair to value (expected return) or maximizing return directly using policy search. UDRL also requires the definition of a command selection strategy, that is, TODO The primary benefit of this formulation lies in its ability to convert a portion of the RL problem into a supervised task.

This allows us to handle part of the complexity of reinforcement learning problems within the supervised learning domain, which is the main area where artificial neural networks are typically applied.

In the UDRL formulation, commands can take different forms; an example is: achieve cumulative reward r in t time steps. The command is defined as a desire-horizon pair (d, h), where d is the desired return, and h indicates in how many timesteps d should be obtained. During training, the model learns to predict what action was taken given the current observation and command (g, h) where g is the actual return obtained.

2.4.9 Deep Q-Network

DQN is a Deep RL algorithm that approximates the optimal action-value function $Q^*(s, a)$ with a neural network. The core idea follows from Q-Learning: if we can obtain a function $Q^* : s \times a \to \mathbb{R}$ that predicts the expected return of taking an action in a given state then it is trivial to obtain the optimal policy π^*:

$$\pi^*(s) = arg \max_a Q^*(s, a)$$

The loss function for DQN minimizes the Temporal Difference error

$$\mathcal{L}_i(\theta_i) = \mathbb{E}_{s,a\sim\rho(\cdot)} \left[(y - Q(s, a; \theta_i))^2 \right],$$

where $y_i = \mathbb{E}_{s'\sim\epsilon} \left[r + \gamma \max_{a'} Q(s', a'; \theta_{i-1}) \right]$, $\rho(s, a)$ is a probability distribution over sequences s and actions a and ϵ is the environment.

2.4.10 Parameter-based Methods

Following the paradigm introduced with Fast Weight Programmers, in parameter-based methods (Sehnke et al. [2010b, 2008b]; Salimans et al. [2017a]; Mania et al. [2018]), at the beginning of each episode, the parameters (weights) of a policy π are sampled from a distribution defined by a hyperpolicy $\nu_\rho(\theta)$ (Faccio et al. [2023]). Therefore, the RL problem goes from finding the parameters θ of a policy π such that the expected return obtained in the environment is maximised, to finding the hyperpolicy parameters ρ that maximise the expected return.

$$J(\rho) = \int_\Theta \nu_\rho(\theta) \int_\mathcal{T} p(\tau|\theta) R(\tau)\, d\tau d\theta.$$

The objective is maximised by computing the gradient of $J(\rho)$ w.r.t. the hyperpolicy parameters:

$$\nabla_\rho J(\rho) = \int_\Theta \int_\mathcal{T} \nu_\rho(\theta) \nabla_\rho \log \nu_\rho(\theta) p(\tau|\theta) R(\tau)\, d\tau\, d\theta.$$

To make the task context-dependent we can rewrite the objective as:

$$J(\rho, c) = \int_\Theta \nu_\rho(\theta|c) \int_\mathcal{T} p(\tau|\theta) R(\tau)\, d\tau d\theta.$$

2.4.11 GoGePo

Goal-Conditioned Generators of Deep Policies (GoGePo) (Faccio et al. [2023]) utilizes an FWP to generate weights of a policy conditioned on a desired return command. The policy is then evaluated by a value function, allowing for optimization of the generator. Given that GoGePo optimizes in parameter space, the evaluator needs to operate in parameter space too. Let $V_w : \Theta \to \mathbb{R}$ be the evaluator function, parametrized by w which estimates the expected return $J(\theta)$ using supervised learning. Then, the objective $\mathcal{L}_G(\rho)$ goes from

$$\mathcal{L}_\mathcal{G}(\rho) = \mathbb{E}_{c\in D, \theta\sim g_\rho(\cdot|c)}[(J(\theta) - c)^2],$$

Where g is the generator, parameterized by ρ and D is the set of possible commands. To:

$$\mathcal{L}_\mathcal{G}(\rho) = \mathbb{E}_{c\in D}[(V_w(G_\rho(c)) - c)^2]$$

which is fully differentiable. Therefore, the parameters of the generator ρ can be learned.

GoGePo uses a probabilistic FWP of the form $g_\rho(\theta|c) = G_\rho(c) + \epsilon$, with $\epsilon \sim \mathcal{N}(0, \sigma^2 I)$ as the hyperpolicy. The RL objective can be written as:

$$J(\rho, c) = \int_\Theta g_\rho(\theta|c) \int_\mathcal{T} p(\tau|\theta) R(\tau)\, d\tau d\theta.$$

2.4.12 Policy Fingerprinting

Policy fingerprinting (Harb et al. [2020]) is a method for evaluating the performance of multiple neural networks through a single evaluator. The idea is to use learnable probing states as inputs to the policy π_θ. The outputs (probing actions) are then combined and given to a multi-layer perceptron, which predicts $V_w(\theta)$. The parameters of the MLP ϕ and those of the learnable probing states form together the evaluator's parameters w. Similarly to techniques where NNs query other trained NNs to improve its performance (Schmidhuber [2015]), the probing states learn to query the policy in ways that reveal its effectiveness. The effectiveness of policy fingerprinting is apparent when dealing with MLPs with one or more hidden layers. Because of symmetries in the weight matrix, identical input-output mappings can arise from numerous equivalent weight configurations. Symmetries often involve permutations of hidden neurons and scaling of weight matrices (Kůrková and Kainen [1994]). Because of their design, the probing actions from fingerprinting are symmetry-invariant.

GoGePo makes uses this method to evaluate policies generated and optimize the generator.

2.5 Autoencoders

Autoencoders (AE) (Ballard [1987]) and Variational Autoencoders (VAE) (Kingma [2013]) are neural network architectures that allow for data compression, reconstruction, and generation. Both models are based on an encoder-decoder architecture but differ in their approach to the latent space: AEs use a deterministic mapping, whereas VAEs use a probabilistic framework.

2.5.1 Standard Autoencoders

Autoencoders are trained with the objective of minimizing the error or difference between the input data and its reconstruction. The error is measured with a loss function, for example, mean squared error. This architecture can be used for different tasks like anomaly detection and denoising. The primary objective of the model is to minimize the reconstruction error, ensuring the decoder can accurately reproduce the input from the latent space representation. The latent space of an autoencoder is not explicitly regularized, leading to a potentially unstructured and sparse latent space that may not effectively capture the underlying structure of the data. When used naïvely, autoencoders are vulnerable to overfitting, as the model tends to memorize the training data without generalizing.

2.5.2 Variational Autoencoders

VAEs attempt to overcome some of the limitations of autoencoders mentioned previously. The model becomes probabilistic, while the core idea remains the same. Instead of mapping input data to a single point in the latent space, the encoder of a VAE generates a probability distribution over the latent space from which the decoder samples a point, which it then decodes. This allows the encoder to learn a more structured latent space from which any point can be sampled and turned into realistic data, which in turn allows for meaningful data generation.

Encoders in Autoencoders output the mean of the latent space representation. In VAEs the encoder now outputs both the mean and variance of the latent space distribution, which is usually modeled as a multivariate Gaussian. To enforce the desired prior over the latent space, we introduce a second term in the loss called the KL divergence. The KL divergence measures the difference between the distribution produced by the encoder (usually a multivariate Gaussian with the learned mean and variance) and the desired prior distribution (often a standard normal distribution). This term ensures that the learned latent space distribution remains close to the prior, encouraging a structured and regularized latent space.

The loss function for a VAE using MSE as reconstruction loss is as follows:

$$L = \mathbb{E}_{q(z|x)} \left[\|x - \hat{x}\|^2 \right] + \mathrm{KL} \left(q(z|x) \| p(z) \right).$$

where:

$$\mathbb{E}_{q(z|x)} \left[\|x - \hat{x}\|^2 \right]$$

is the reconstruction loss; x is the original input, $\hat{x}$ is the reconstructed output, and $\|x - \hat{x}\|^2$ is the MSE;

$$\mathrm{KL} \left(q(z|x) \| p(z) \right)$$

is the KL divergence term; $q(z|x)$ is the encoder's output distribution and $p(z)$ is the prior distribution.

Chapter 3

Related Works

The development of UDRLPG is based on several established concepts within the field of RL, specifically those that learn directly or search in the policy parameter space. This section reviews the relevant works that provide the foundations for UDRLPG.

3.1 Fast Weight Programmers

Fast Weight Programmers (FWPs) introduced the concept of using one NN to output weight updates for another target NN (Schmidhuber [1992, 1993]). Later, the same concept was extended to deeper NNs under the name Hypernetworks (Ha et al. [2016]). FWPs allow for dynamic context-dependent weights, making the weights of the NN non-static after training. Linearized self-attention mechanisms (linear transformers) are equivalent to fast weight controllers where information is stored using sequences of additive outer products of keys and values (Schlag et al. [2021]; Schmidhuber [1992]). In our approach, FWPs are utilized in the context of RL to generate policies conditioned on a return command. However, FWPs find applications in various domains, such as memory-based meta learning Schmidhuber [1993]; Miconi et al. [2018]; Gregor [2020]; Kirsch and Schmidhuber [2021]; Irie et al. [2022]; Kirsch et al. [2022], predicting parameters for unseen deep architectures (Knyazev et al. [2021]), and RL (Gomez and Schmidhuber [2005]; Najarro and Risi [2020]; Kirsch et al. [2022]).

3.2 Parameter-based Value Functions

Parameter-based value functions (PBVFs) (Faccio et al. [2021]; Harb et al. [2020]) abstract the concept of traditional value functions by learning an estimate of the expected return conditioned on the policy parameters. In RL, value functions typically estimate the expected return given a state $V(s)$ or state-action pair $Q(s, a)$. Parameter-based value functions $V(\theta)$ estimate the expected return for a given policy parameter vector θ, capturing a direct relationship between policy parameters and return. PBVFs $V(\theta)$ learn to map policy parameters to their expected return in a given environment. This enables evaluation and adjustment directly in parameter space rather than relying solely on updates in action space. Given a function $V(\theta) = \hat{r}$ that maps policy parameters to its expected reward, we can invert this mapping to determine the parameters θ given an expected reward r, which is the core idea of UDRLPG.
A policy-conditioned value function

$$\hat{r} = V(\theta)$$

approximates the inverse map of UDRLPG

$$\theta = V^{-1}(c) = G(c),$$

where c is the command representing the expected reward of the decoded policy and G is the generator of policies.

3.3 GoGePo

The evaluator function $V_w : \Theta \to \mathbb{R}$ is at the core of GoGePo, as it allows for the optimization of the generator. UDRLPG does not require an evaluator to train its generator. GoGePo's generator learns to minimize the difference between the return command and the estimated return of the generated policy provided by the parameter-based evaluator. UDRLPG's generator learns instead to minimise the error between a known policy achieving the return on which the generator was conditioned and the generated policy.

3.4 Evolution Strategies

Evolution Strategies are population-based optimization algorithms inspired by natural evolution. They directly explore the policy parameter space by generating a population of samples with different policy configurations, evaluating their performance utilizing a fitness function, and then using those evaluations to select the best subset of individuals and guide the search toward optimal policies. Evolution Strategies demonstrate the effectiveness of directly exploring policy space, showing that optimizing in parameter space can overcome some of the limitations associated with action-space methods, such as sparse reward signals Salimans et al. [2017b].

3.5 Policy Gradients with Parameter-Based Exploration

In policy gradient like REINFORCE, the policy parameters θ define a probabilistic policy $\pi_\theta = p(a|s, \theta)$. In this setting, trajectories can be sampled by choosing actions according to the distribution produced. The gradient can then be computed w.r.t the policy parameters. However, sampling from the policy at every timestep leads to high variance in the samples which translates into high variance in the estimated gradient(Sehnke et al. [2008a]).

Policy Gradients with Parameter-Based Exploration (PGPE) (Sehnke et al. [2008a]) addresses the problem by replacing the policy with a probability distribution over the parameters.

$$p(a_t|s_t, \rho) = \int_\Theta p(\theta|\rho)\delta_{F_\theta(s_t),a_t}\partial\theta,$$

Where ρ are the parameters defining the distribution over the policy parameters θ,$F_\theta(s_t)$ is the deterministic action chosen by the model, and δ is the Dirac function. Since the actions are now deterministic, a trajectory can be obtained from one sample of the parameters. Reducing the variance of the gradient estimate.

3.6 Upside Down Reinforcement Learning

UDRL transforms the RL problem into a supervised learning task by learning a mapping between command and action (Schmidhuber [2019]; Srivastava et al. [2019]). The data utilized to train the model can be collected in an on-line fashion as the model improves (Srivastava et al. [2019]; Zheng et al. [2022]) or offline (Janner et al. [2021]; Chen et al. [2021]). UDRLPG stems from the shift in perspective proposed by UDRL. In our approach, reward commands guide the generation of policy parameters; instead of optimizing the policy to achieve a desired command in action space, UDRLPG trains a generator to map or decode commands to policy parameters, thus optimizing in parameter space.

It has been shown that in stochastic environments with episodic resets, some UDRL variants can fail to maximize the probability of satisfying their commands. (Štrupl et al. [2022])

Chapter 4

Method

4.1 Motivation

In reinforcement learning, the standard approach to policy optimization typically involves directly training a policy network to maximize expected returns. However, this often requires complex architectures, such as those used in actor-critic methods where both the policy (actor) and the value function (critic) are simultaneously optimized. Although these methods have succeeded significantly in various domains, they require constant coordination between the actor and the critic. This coordination issue is further compounded by the need to estimate the value function accurately. This task can become increasingly difficult as the complexity of the environment grows.

Recent approaches, such as GoGePo (Goal-Conditioned Generators of Deep Policies), have attempted to shift the focus away from from optimizing the policy directly. GoGePo adopts a generative approach, learning a generator that produces policies based on different target returns or goals. However, this approach still relies on an additional evaluator or critic network that assesses the quality of the generated policies, introducing additional complexity, as the evaluator must be jointly optimized with the generator, leading to potential stability, overfitting, and computational overhead challenges.

The motivation for this thesis lies in simplifying this approach introduced with GoGePo by removing the need for a separate evaluator. Instead of relying on a generator-evaluator pair, the proposed method focuses on learning a single, unified policy generator or decoder capable of producing policies that can achieve any desired return. The underlying idea is that, by conditioning the generator on the desired return directly, the system can learn to produce policies capable of achieving specific rewards without needing an external critic, allowing the streamlining of the learning process by reducing the architectural complexity.

A significant challenge with this method is that a single command c can correspond to various behaviors or policies, indicating that the target function to learn could exhibit multimodality, making it harder for a maximum likelihood method to represent the data's variability effectively. A way to work around this complexity when optimizing in parameter-space is to utilize a parameter-space evaluator. Evaluating a policy in parameter space is a remarkably complex task that GoGePo solves using policy fingerprinting, a type of parameter-based value function.

By removing the need for the evaluator, the proposed method is less prone to issues arising from misalignment between the actor and critic, reducing the risk of biased or inaccurate value estimates, which are common problems in actor-critic methods. As the generator would not depend on potentially noisy or inconsistent feedback from an evaluator, this may contribute to improved learning outcomes at the cost of higher variance.

The removal of the evaluator means that objective of the generator in UDRLPG must change. In GoGePo the generator is trained to minimize the error between the return command and the return of the generated policy estimated by the evaluator. The UDRLPG is trained to minimize the mean squared error between the generated policy and a policy known to achieve the return requested to the generator.

4.2 UDRLPG

We develop UDRLPG, our algorithm to generate policies that achieve any desired return. In contrast to more classic deep-RL algorithms, UDRLPG is a parameter-based method (Sehnke et al. [2010a] Salimans et al. [2017a]Mania et al. [2018]). As such, it directly optimizes over the space of policies rather than individual actions. UDRLPG learns to decode a given command into weights of a target NN. The command is an encoding of the desired return. The target weights generated by the decoder are those of a policy $\theta \in \Theta$ that, when evaluated in the chosen environment, will obtain the requested return. The decoder is represented by a type of feed-forward Fast Weight Programmer (FWP) known as hypernetworks Ha et al. [2016]. Formally, the FWP is a function $G_\rho : \mathbb{R}^{n_c} \to \Theta$, where $c \in \mathbb{R}^{n_c}$ is the context or command and ρ are the parameters of the FWP. In particular, for exploration purposes, we consider a non-deterministic FWP $g_\rho(\theta, c) = G_\rho(c) + \epsilon$ where ϵ represents Gaussian noise $\epsilon \sim \mathcal{N}(0, \sigma^2 I)$. The FWP learns to minimize the error

$$\mathcal{L}_G(\rho) = \mathbb{E}_{c \in D}[(G_\rho(c) - \theta^*(c))^2],$$

Where D is the replay buffer, c are the returns of policies present in D, $\theta^*(c)$ is a policy with expected return c.

The detailed algorithmic description of the process is described in Algorithm 1. The algorithm consists of two main steps. In the first stage, the hypernetwork (or generator) undergoes an iterative gradient update to refine its policy generation capabilities. This involves sampling from the replay buffer that stores previously generated policies along with their corresponding expected returns. How the policies are sampled from the buffer is a critical step to ensure stable training; more details will be discussed in the dedicated subsection. The sampled returns are fed into the hypernetwork, which attempts to generate policies that achieve the specified returns. The generator is trained to minimize the mean squared error between the generated and sampled policy

$$\mathcal{L}_G(\rho) = \mathbb{E}_{(r,\theta) \in D}[(\theta - G_\rho(r))^2].$$

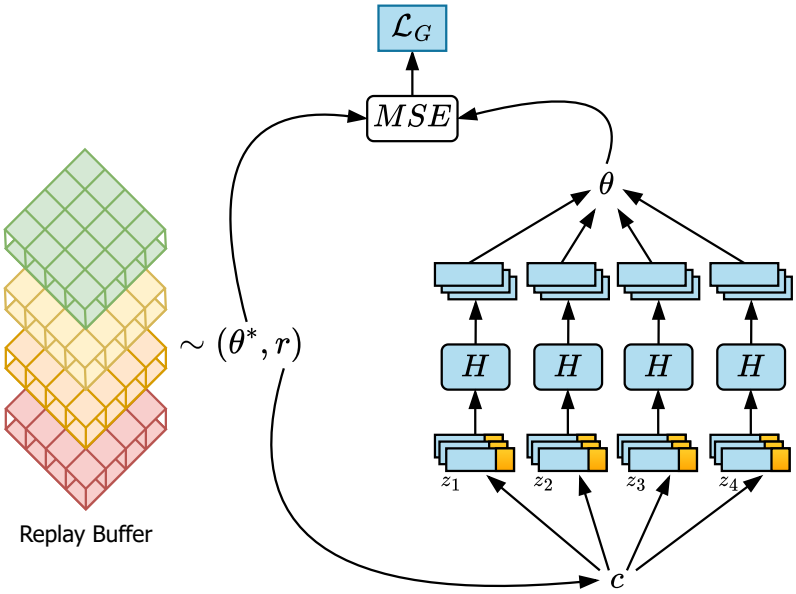
This iterative optimization process gradually improves with gradient descent the generator's ability to produce policies achieving the desired returns. Once the generator has been optimized, the second stage involves generating new policies using its trained weights. The generator requires a command as input to produce a policy. The strategy used to select this command is an important aspect of the algorithm. Typically, it involves requesting the highest return observed in the buffer. Asking for a return that is too far from what the generator has been trained on leads to instability. Requesting the highest return observed ensures that the learning process is moving forward while maintaining stability. The generated policy, however, is not used as-is. Noise is added to the policy to introduce exploration. This noise is sampled from a Gaussian distribution $\mathcal{N}(0, \sigma^2 I)$, where σ is a hyperparameter that controls the extent of the perturbation applied to the policy. The introduction of noise is vital for the learning process as it allows the algorithm to explore a wider range of potential policies rather than getting stuck in local optima. The perturbed policy is then evaluated by deploying it in the environment. The environment provides feedback in the form of a return. The resulting trained model is capable of generating policies achieving any arbitrary return in a zero-shot manner, without any additional training. Figure 4.1 shows a visual representation of the update step in UDRLPG.

It is worth noting that initially, this method's architecture was a Conditional Variational Autoencoder (CVAE) (Sohn et al. [2015]). A trained CVAE with sufficient regularization makes the latent space well-structured, allowing the decoder to generate meaningful outputs by giving Gaussian noise and the command as input. This architecture would have allowed for the generation of different policies, achieving the same return. However, early results have shown that the method suffered from posterior collapse (Bowman et al. [2015]), that is, the decoder would effectively learn to ignore the latent space

4.2.1 Exploration and expoitation

In any reinforcement learning algorithm, finding a sensible balance between exploration and exploitation is crucial. This principle holds in UDRLPG. The core challenge in RL is to exploit existing knowledge of high-reward policies to achieve the best possible outcomes while simultaneously exploring the policy space to discover potentially superior strategies that still need to be identified. Finding a balance is fundamental for any algorithm, as focusing too much on exploitation can lead to suboptimal performance

Figure 4.1. Visualization of update step in UDRLPG



Algorithm 1 UDRLPG

$D \leftarrow$ empty replay buffer with buckets
following some strategy add n random policies to D
$G_\rho \leftarrow$ Differentiable generator (decoder)
while ending condition not met **do**
    for j in *update_repeats* **do**
        sample a batch $b = \{(r, \theta)\}$ from D
        Update generator by stochastic gradient descent: $\nabla_\rho \mathcal{L}_\rho(b_r, b_\theta)$
    end for
    for j in *rollout_repeats* **do**
        pick a desired return r_0
        sample policy parameters $\theta \sim G_\rho(\theta | r_0)$
        sample noise $\epsilon \sim \mathcal{N}(0, \sigma^2 I)$
        $\theta \leftarrow \theta + \epsilon$
        $r \leftarrow$ return from simulating π_θ in the environment for one episode
        add (r, θ) to D
    end for
end while

by converging to a local maximum, while excessive exploration may impede convergence. In UDRLPG, exploration is performed by perturbing the policies generated during the rollout step. The perturbation of the policy is done by adding Gaussian noise to the generated policy. The noise is sampled from $\mathcal{N}(0, \sigma^2 I)$ where the amount of the noise applied to the policy is controlled by σ, the standard deviation of the normal distribution. In UDRLPG σ is a hyperparameter that dictates the level of randomness or variability introduced when generating new policies. A higher value of σ increases the degree of exploration by allowing the algorithm to venture into less-explored areas of the policy space. Higher σ values can be beneficial in complex environments where deceptive local optima may surround the global optimum, and broader exploration can jump out of local optima. In contrast, a lower σ reduces the amount of exploration, thereby favoring exploitation. When σ is low, the algorithm focuses more on the policies that have already demonstrated high rewards, incrementally pushing their performance towards the local optimum within the current landscape. This exploitation is critical for the algorithm to achieve consistently high performance by leveraging known information. During the rollout step, the generator is asked to produce a policy conditioned on the highest reward seen, ensuring that the algorithm continually prioritizes policies that yield better outcomes, driving it toward increasingly optimal performance. However, without the injection of noise through σ, the algorithm will converge too quickly on a suboptimal solution, failing to explore alternative strategies that could lead to higher rewards. The noise, therefore, prevents premature convergence to lower reward policies. Perturbing the policy is how exploration happens in UDRLPG, and the noise parameter σ controls the amount of exploration.

4.2.2 Replay buffer and buckets

In deep reinforcement learning, the replay buffer is a crucial component designed to store and manage past experiences, enabling efficient learning by reusing past data and providing a mechanism to break the temporal correlations in sequential data. Traditionally, in model-free RL methods such as Q-learning or deep Q-networks (DQN), the replay buffer stores tuples of experiences in the form of a tuple of a form similar to state, action, reward, and next state, which are then sampled during training following some strategy to update the policy. This process allows the agent to learn from diverse experiences, improving stability and convergence. In parameter-based methods, the concept of a replay buffer is extended to store entire sets of policy parameters along with their corresponding expected returns. Instead of storing individual experiences, this type of replay buffer holds snapshots of the entire policy's weights, paired with the cumulative reward obtained by following that policy during the episode.

During training, the replay buffer plays a significant role, as the policies selected from the buffer to compute the gradients in order to update the parameters of the generator inevitably alter the learning process. Training the generator with a sampling strategy that prioritizes low-reward policies will slow down the learning of higher-reward policies and eventually lead to stagnation caused by the overfitting of such low-performing policies, which in turn results in the generator being unable to produce higher-reward policies. On the other hand, performing updates by sampling mostly high-reward policies from the buffer will result in fast convergence to high-reward policies. However, the generator will quickly lose the ability to produce lower-reward policies.

Typically, the replay buffer is either temporally sorted , meaning the order of the buffer is the same as the insertion order, or it is sorted by reward. Sampling can be performed uniformly, where every element in the buffer has the same probability of being chosen, or by applying weighting on the elements of the buffer, increasing the chance of choosing some elements in specific positions of the buffer. Higher weights are usually applied to the part of the buffer where the high-reward policies lay such that the likelihood of such policies being chosen increases. If the buffer is sorted in increasing reward order, then the section at the end of the buffer would contain the high-reward policies. The same applies to temporally sorted buffers, only if we assume that increasingly higher rewards are sought during the rollout step (when new policies are generated). In this case, we could apply any weighting strategy on the buffer, such as exponential weighting, to increase the chance of choosing the higher reward policies in the buffer.

Sampling uniformly from the buffer leads to slower converge to high-reward policies compared to weighted strategies because the focus is evenly distributed throughout the buffer. With uniform sampling, in situations where the buffer is mostly filled with low-reward policies, learning can stagnate due to insufficient focus on the few high-reward policies. In the case of temporally sorted buffers with weighting, learning stagnation can also occur when the generator cannot learn quickly to produce better-performing policies than the ones currently in the buffer, causing the hypernetwork to overfit to low-reward policies

and severely hindering the learning process. Even when forcefully prioritizing better policies by sorting the buffer and applying a weighting strategy, if the number of low-reward policies is much greater, we can still experience overfitting of low-reward policies or, if the weighting on high-reward policies is exceptionally high, low-reward policies will be quickly forgotten by the network. Learning stagnation is a significant issue when training policy generators independently of the sampling and sorting strategy because sampling probability heavily depends on the absolute number of policies present in the buffer.

To mitigate this issue, the proposed algorithm organizes the replay buffer into performance-based buckets containing policies within a specific return range. This method decouples the sampling probability from the absolute number of policies in each performance category, ensuring a more balanced representation of high and low-reward policies during training. As a result, even if the buffer is populated with a large number of suboptimal policies, the sampling process can still adequately represent the higher-performing policies, reducing the risk of getting stuck in suboptimal training phases and improving the robustness and effectiveness of the generator's learning process. Buckets mitigate the issues that arise in a classic replay buffer when it is populated with too many low-return policies because, with buckets, the sampling becomes independent of the number of policies. As long as a given bucket contains more than one policy it will be considered for sampling with a probability that is directly defined by the weighting strategy, rather than the sheer count of policies within it. This means that each cluster of similar return policies contributes to the training process proportionally to the defined weighting strategy and independently of how many of those policies with similar returns are present in the buffer, ensuring a more diverse and effective training process.

The bucket system can be defined as dynamic because the number of buckets is allowed to increase during training. The buckets are initialized they are created as evenly spaced containers based on the defined number of buckets and the given boundaries for maximum and minimum return r_{min} and r_{max}. If a policy with return $r_{new} > r_{max}$ is obtained during training, n new buckets will be added until the one whose range contains r_{new} is added. For example, consider a toy environment with minimum return 0 and an unknown maximum return r_{max} of which we only have an approximation of 10. We set the boundaries for the buckets as $[0, 10]$ and choose to have 5 buckets. The first bucket will contain all policies achieving return $r \in [0, 2)$. Assume that during training, we produce a policy that obtains a return of 13. If we placed that policy in the last bucket the buckets would not be representing a fixed range of policies anymore. To avoid this problem, additional buckets are created, following the spacing originally defined spacing (2 for this example). Therefore two new buckets would be created, one to house policies with reward $r \in [10, 12)$, which will be empty, and the second containing $r \in [12, 14)$ containing the new policy. Figure 4.2 represents a visualization of this example.

Algorithm 2 Sampling from replay buffer with performance buckets

1: **Input:** nonempty replay buffer D; batch size s
2: **Output:** Batch B of size s
3: $D_{masked} \leftarrow D$ with masked empty buckets
4: $b \leftarrow s$ buckets selected from D_{masked} with replacement following a weighting strategy
5: $B \leftarrow$ randomly selected policy from each bucket in b
6:

When dealing with buckets, choosing the correct weighting strategy is just as crucial as when dealing with classic replay buffers. By using performance-based buckets, the proposed algorithm maintains a more consistent balance, focusing on learning from policies that are more likely to contribute positively to the overall performance while still allowing occasional updates on lower-performing policies. This enhances the stability of the learning process, ultimately leading to a more efficient and effective optimization of the generator. The weighting strategy, as well as its scaling and the number of buckets, are all hyperparameters of UDRPG. The sampling process with buckets is defined in Algorithm 2

4.3 Observation normalization

Observation normalization (Mania et al. [2018]; Faccio et al. [2021, 2023]) is a technique used to ensure equal influence of the different components in the observation vector during training. Say that we have a toy environment with an observation vector composed of two components $a \in [1, 10]$ and $b \in [100, 1000]$.

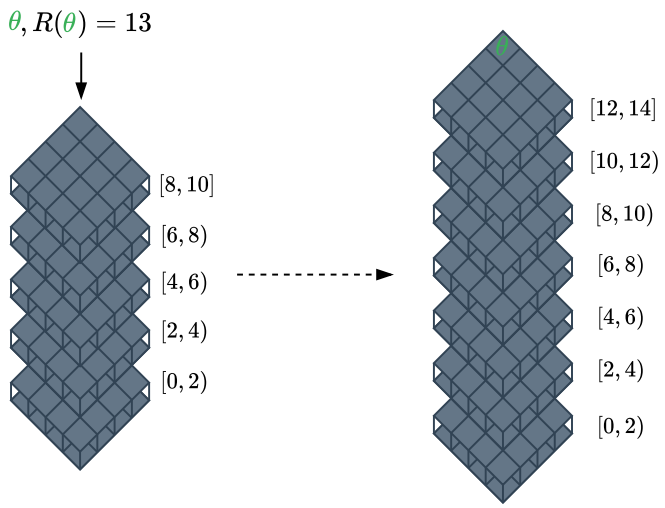


Figure 4.2. Visualization of dynamic buckets

Then, the sensitivity to changes in b will be higher compared to a. In other words, proportionally identical changes in the two states lead to disproportional changes in the output of the policies, i.e., the actions. If we normalize the states, we ensure that all components of the observation have a similar impact on the output. Before dividing the difference between observation and mean, a small quantity is added to the standard deviation to avoid division by zero. The observations used to compute the running mean and running standard deviation are obtained only from interactions with the environment during training.

4.4 Hypernetworks

In our approach, the hypernetwork generates the parameters θ of the target network by dividing them into smaller slices s_l. A shared neural network H outputs

$$s_l = H_\phi(z_l, c),$$

where parameters ϕ are the parameters of the NN H, z_l is a learned embedding, and c is additional conditioning that can be either scalar or vector-valued. In this case, it is a scalar representing the requested return. θ is then constructed by concatenating all the generated slices: $\theta = [s_1, s_2, s_3, \ldots]$. Our specific implementation follows the one of GoGePo, and is tailored to generate policy parameters of multi-layer perceptrons (MLPs), θ which consists of a series of weight matrices K^j. For each policy layer $j \in \{1, 2, \ldots, n_K\}$, where n_K is the number of layers, the HN H_ϕ generates each slice of each weight matrix. Specifically, the generator G_ϕ splits each weight matrix into smaller slices $s^j_{mn} \in \mathbb{R}^{f \times f}$, where j refers to the policy layer and m, n index the slices within the weight matrix of layer l. For every slice, a small embedding vector $z^j_{mn} \in \mathbb{R}^d$ is learned. The network H_ϕ is an MLP whose output is reshaped from a vector of size f^2 into an $f \times f$ matrix:

$$s^j_{mn} = H_\phi(z^j_{mn}, c).$$

These slices are concatenated over two dimensions to form the full-weight matrix:

$$K^j = \begin{bmatrix} s^j_{11} & s^j_{12} & \cdots \\ s^j_{21} & s^j_{22} & \cdots \\ \vdots & \vdots & \ddots \end{bmatrix}.$$

The complete hypernetwork generator G_ϕ is composed of the shared network H_ϕ and all embedding vectors z^j_{mn}, with its learnable parameters being $\rho = \{\phi, z^j_{mn}\ \forall m, n, j\}$. The conditioning c is appended to each learned slice embedding z^j_{mn}, and the concatenated vectors are then fed into the network H_ϕ which outputs the weight slice. To allow for flexibility in input and output layers, which are dependent on environmental observations and actions (input and output of the target network), two separate networks H_i and H_o are utilized for these layers.

4.5 Differences with GoGePo

At its core, UDRLPG effectively streamlines the method initially proposed by GoGePo. Both GoGePo and UDRLPG are, in essence, policy generation algorithms that employ a probabilistic Fast Weight Programmer (FWP) in the form of a hypernetwork as the policy generator, which is conditioned on a given return command representing the expected return of the generated policy. In the case of UDRLPG, the only learnable parameters are ρ, which are the parameters of the FWP itself. GoGePo, instead, involves the simultaneous optimization of two distinct architectures: the generator and the evaluator. The latter component is not present in UDRLPG; the evaluator's role is to learn to estimate the performance of a policy. Its feedback is subsequently utilized to update the gradients of the generator. The use of an evaluator circumvents the possible issues arising from the multimodality issue. UDRLPG optimizes the generator by minimizing the distance in weight space between policies known to produce the requested return and the generated policy. This simplification makes it necessary to increase stability during the training process, achieved through specific improvements, such as performance-based buckets within the replay buffer. GoGePo employs weight scaling, a heuristic method involving scaling the generator's output so that it remains close to the default initialization of the target network, from which UDRLPG does not gain any benefit. During the generation of new experience, GoGePo conditions the generator on a slightly higher return than the highest in the buffer, allowing for faster convergence; this is not required in UDRLPG.

Chapter 5

Experimental Setup

5.1 Environments

This section provides a detailed description of the environments used to evaluate UDRLPG. A diverse set of control environments was selected from the Gymnasium library (Towers et al. [2024]), namely `InvertedPendulum-v4`, `Swimmer-v4`, and `Hopper-v4`. These environments were selected due to their escalating complexity, extensive use in the literature, and the ability to challenge and demonstrate the capabilities of RL algorithms effectively (Duan et al. [2016]).

5.1.1 InvertedPendulum-v4

The Inverted Pendulum environment is a widely studied basic task (Duan et al. [2016]) in reinforcement learning. The task involves balancing a pole, which is free to pivot at one end, on a cart that can move horizontally along a track. The agent must apply a force to the cart to keep the pole upright. Despite its apparent simplicity, the Inverted Pendulum is a challenging environment because the agent must learn to balance the pole in the unstable upright position, where minor deviations can lead to a rapid increase in the angle and cause the pole to fall, requiring the RL algorithm to learn a precise control policy that can respond accurately to small perturbations. The observation space is continuous and consists of the cart's position along the linear surface, the vertical angle of the pole on the cart, the linear velocity of the cart, and the angular velocity of the pole on the cart. The agent receives a vector representing these variables and must output a continuous action representing the force applied to the cart. The reward function simply consists of a positive (+1) reward for each time step that the pole is kept upright.

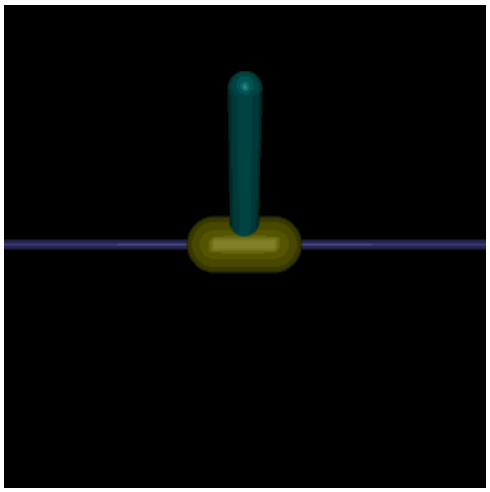


Figure 5.1. Illustration of InvertedPendulum-v4. Source: Gymnasium documentation

5.1.2 Swimmer-v4

The Swimmer environment simulates a robotic swimmer moving through a fluid. The goal is for the agent to control the torques applied to the joints of a segmented body to propel itself forward as quickly as possible. The observation space of the Swimmer environment is continuous. It includes eight variables, including the positions and velocities of the segments that make up the swimmer's body and the velocities of the joints. The agent controls the torques applied to the joints between these segments. The environment is modeled after fluid dynamics principles, where the movement is affected by drag forces from the surrounding fluid, making it a challenging control problem. This environment requires the agent to learn complex, coordinated motor patterns to produce effective swimming motions. It is a good choice for evaluating the ability to learn policies where good-performing strategies involve multiple coordinated actions. The reward structure in this environment rewards the agent for forward velocity, encouraging the development of efficient swimming strategies.

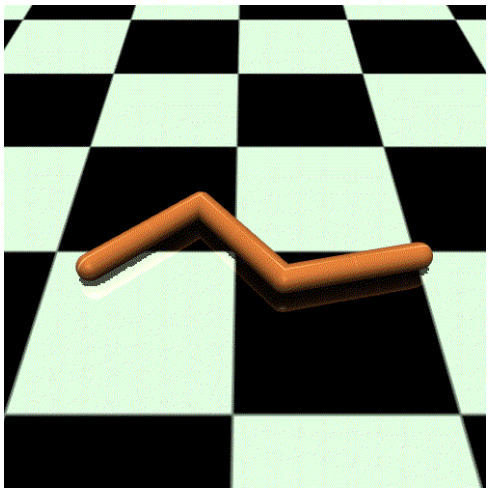


Figure 5.2. Illustration of Swimmer-v4. Source: Gymnasium documentation

5.1.3 Hopper-v4

The Hopper environment presents the most complex and dynamic challenge for evaluating UDRLPG. The agent must control a one-legged robot to hop forward without falling. This environment is a standard benchmark for testing RL algorithms on tasks that involve balance, coordination, and continuous control. The Hopper environment's observation space comprises eleven variables, including joint angles, angular velocities, and the horizontal and vertical positions of the robot's center of mass. The agent controls the torques applied to the joints at the leg, foot, and thigh. This environment is particularly challenging because the agent must maintain balance while generating forward motion. The agent must learn to apply the correct sequence of actions to achieve stable movement. This environment assesses the RL algorithm's capability to handle more intricate dynamics than simpler environments like InvertedPendulum. It involves learning to balance while moving and managing the interactions between different joints. The reward structure typically provides positive reinforcement for forward motion and penalizes the agent for falling or making inefficient movements. It also includes a positive reward for survival, which often leads to training getting stuck into a local optimum in which the agent simply learns to stay still, for this reason, the survival bonus for this environment will be removed during training of UDRLPG.

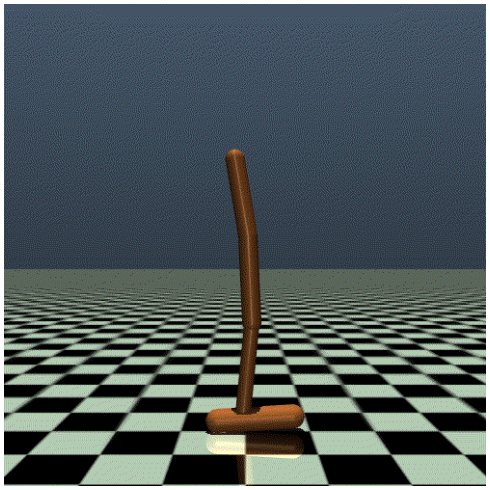


Figure 5.3. Illustration of Hopper-v4. Source: Gymnasium documentation

5.2 Evaluation

UDRLPG is evaluated systematically to assess its performance across the chosen environments. Considering the model's capability to generate policies covering a range of returns rather than solely maximizing them, in addition to the traditional evaluation process that examines the best performance achieved by the model, a more extensive evaluation is necessary to comprehend its performance across the entire reward spectrum.

During training, at specific timesteps, two evaluation routines are run to understand the model's performance: one designed to measure the model's current ability to generate policies that maximize rewards and the other designed to assess its robustness in generating policies over a broad spectrum of requested rewards.

5.2.1 Evaluation

This evaluation is performed at each 1% increment of the total timesteps throughout the entire training process. The purpose of this evaluation is to gain insight into the algorithm's learning process, specifically to understand if the model is learning to produce higher and higher rewards over time. As training examples for a given reward start populating the buffer, the model should start to learn to produce policies achieving such return.

The return is determined by first generating five distinct policies conditioned on the maximum reward observed by the buffer, and then calculating the mean of their cumulative rewards.

Judging the performance of the model during training by only considering a single evaluation may not capture the true performance

A single evaluation datapoint may not exhaustively represent the training process up to a given timestep, as it can be very sensitive to fluctuations. For this reason, a more exhaustive metric can be obtained by averaging the last 20% of the evaluations. This metric is a better indicator of performance over time, as it offers a clearer and more reliable indication of the model's performance by focusing on the model's recent behavior rather than at a single point in time, smoothing out short-term fluctuations and noise that may occur in the training process.

5.2.2 Identity Evaluation

Given the nature of UDRLPG, we must also monitor the model's ability to produce policies across a broad spectrum of returns. For this reason, at 10% intervals of the total timesteps, a more comprehensive evaluation is performed, aimed at assessing the generator's capability to produce policies across a broad spectrum of reward conditions.

During this evaluation routine, the model is asked to produce policies conditioned on a series of evenly spaced return commands, ranging from the minimum to the maximum achievable return in the environment. For each of these target returns, the model's performance is measured by comparing the requested return with the average return obtained by ten independently generated policies. This comparison is typically visualized on a plot of requested versus achieved returns. Optimal performance would be indicated by achieved return lying along the identity line, meaning the achieved returns exactly match the requested ones.

Unlike the previous evaluation method, analyzing this metric also provides insight into the model's generalization ability because the return on which the model is conditioned is not guaranteed to be present in the replay buffer. A model with flawless performance would produce a linear relationship where, for every requested return, the model produces a corresponding policy that achieves that exact return. Deviations from the identity line reveal the extent of the model's imperfections, such as tendencies to underperform or overperform relative to the requested return or difficulties in generalizing to certain return levels. This deviation can be measured by computing the mean square error between the identity line and the line produced by the evaluation process to obtain a score indicating how well the generator fits this line.

It should be mentioned that in some environments achieving every intermediate return can be difficult or even impossible. This can occur, for instance, when the optimal policy is simpler than a slightly suboptimal one, when the optimal policy is very similar to a policy with minimal expected return, or when a high reward is given upon reaching the goal state.

5.3 Hyperparameter tuning

To optimize the performance of UDRLPG, hyperparameter tuning was approached in multiple stages. The initial stage involved exploring a broad search space using random searches, simultaneously sweeping over an extensive set of hyperparameters. This approach aimed to identify a set of hyperparameters that yielded reasonable performance as a baseline for more refined tuning. The performance of each hyperparameter configuration was evaluated using the "last average evaluations" metric, as described in Section 5.2.1.

Once a set of promising, albeit suboptimal, hyperparameters was determined, a more targeted tuning process began. This refined tuning involved performing sequential grid searches over individual hyperparameters. The grid search process was designed to focus on one hyperparameter at a time while keeping the others fixed to better understand each hyperparameter's impact on the overall performance. Given that hyperparameters may exhibit dependencies — where the optimal value of one hyperparameter may change in response to the value of another — this approach required careful attention and iterative adjustments. To ensure statistical significance and reliable results, the performance of each unique hyperparameter configuration in a sweep was evaluated by averaging the results over multiple runs with different random seeds. The number of runs per configuration was gradually increased from an initial set of three to as many as twenty in the later stages.

The initial random search stage was conducted on InvertedPendulum, the simpler of the chosen environments, to establish a baseline of usable hyperparameters. This allowed for efficient exploration in a less complex context before progressing to more challenging environments. Afterward, tuning efforts were carried out primarily on Hopper and Swimmer, the more complex environments. Only during the very last stages of the tuning were all three selected environments tested simultaneously, ensuring that the chosen hyperparameters provided robust performance across diverse environments, reflecting the generalizability and adaptability of the proposed algorithm. Table 5.1 shows the final hyperparameters for UDRLPG. Hyperparameters are fixed across environments.

5.4 Comparison Algorithms

To evaluate the performance of the proposed algorithm UDRLPG, we selected GoGePo and Deep Deterministic Policy Gradient (DDPG) (Lillicrap [2015]) as comparison baselines. GoGePo was chosen as it shares the foundational concept with UDRLPG, both being policy generators; since UDRLPG is a simplification of the approach used in GoGePo, it serves as an obvious benchmark for assessing the

effectiveness of our proposed method. DDPG was elected due to its recognition as a popular policy gradient algorithm, positioning it as a fitting benchmark for assessing UDRLPG against established RL algorithms. Table 5.2 and 5.3 show the hyperparameters used for GoGePo and DDPG respectively.

Hyperparameter	**UDRLPG**
Policy Architecture	MLP, 2 hidden layers, 256 neurons each, with bias
Policy Nonlinearity	tanh
Value Function Architecture	MLP, 2 hidden layers, 256 neurons each, with bias
Batch Size	20
Optimizer	Adam
Learning Rate Generator	5e-6
Exploration Noise Scale	0.1 (in parameter space)
Number of Generator Updates	12
Number of Rollout Steps	1
Replay Buffer Size	10k
Number of buckets in replay buffer	100, Dynamic
Replay buffer weighting	Exponential
Replay buffer weighting scaling	0.04
Size of slices s^j_{mn} produced by the hypernetwork	16 x 16
Warm-up policies	10
Size of learnable hypernetwork embeddings z^j_{mn}	8
Observation Normalization	True
Survival Reward Adjustment	True only for Hopper
Environmental interactions	100k for InvertedPendulum-v4; 3M for all other environments

Table 5.1. Hyperparameters for UDRLPG

Hyperparameter	**GoGePo**
Policy Architecture	MLP, 2 hidden layers, 256 neurons each, with bias
Policy Nonlinearity	tanh
Value Function Architecture	MLP, 2 hidden layers, 256 neurons each, with bias
Batch Size	20
Optimizer	Adam
Learning Rate Generator	2e-6
Learning Rate Value Function	5e-3
Exploration Noise Scale	0.1 (in parameter space)
Number of Generator Updates	20
Number of Value Function Updates	5
Number of Probing States	200
Initialization of probing states	Uniformly random in [0, 1)
Replay Buffer Size	10k
Replay buffer weighting	True, weights $1/x^{1.1}$
Replay buffer weighting scaling	0.04
Size of slices s^j_{mn} produced by the hypernetwork	16 x 16
Size of learnable hypernetwork embeddings z^j_{mn}	8
Survival Reward Adjustment	True only for Hopper
Observation Normalization	True
Environmental interactions	100k for InvertedPendulum-v4; 3M for all other environments

Table 5.2. Hyperparameters for GoGePo

Hyperparameter	**DDPG**
Policy Architecture	MLP, 2 hidden layers, 256 neurons each, with bias
Policy Nonlinearity	ReLU
Steps Per Epoch	4000
Epochs	100
Replay Buffer Size	1000k
Gamma	0.99
Polyak	0.995
Learning Rate Actor	1e-3
Learning Rate Value Function	1e-3
Batch Size	100
Start Steps	10k
Update After	1000
Update Every	50
Exploration Noise Scale	0.1 (in action space)
Survival Reward Adjustment	False
Observation Normalization	False
Environmental interactions	100k for InvertedPendulum-v4; 3M for all other environments

Table 5.3. Hyperparameters for DDPG

Chapter 6

Results

We present the results of the evaluation of UDRLPG in comparison to two baseline algorithms: GoGePo and DDPG. For each environment, we analyze the mean return as a function of the environment interactions, the minimum maximum, and the variance of the last returns. Finally, we analyze the model's ability to produce policies across the reward spectrum.

6.1 InvertedPendulum-v4

The results presented in Figure 6.1 illustrate the performance of UDRLPG in comparison to the baseline algorithms GoGePo and DDPG during training in InvertedPendulum-v4. The learning curves indicate that UDRLPG converges to final mean return values comparable to the baselines, with its performance stabilizing around a similar range. UDRLPG shows a slower learning curve compared to the baselines. Figure 6.2 shows the variance of the final returns across 20 independent runs. UDRLPG has a broader distribution compared to GoGePo, indicating greater variability in final returns. Table 6.1 quantifies these observations, showing that UDRLPG is also capable of achieving the maximum reward of 1000 for the environment in question. Figure 6.3 shows the method's ability to generate policies across the spectrum of rewards.

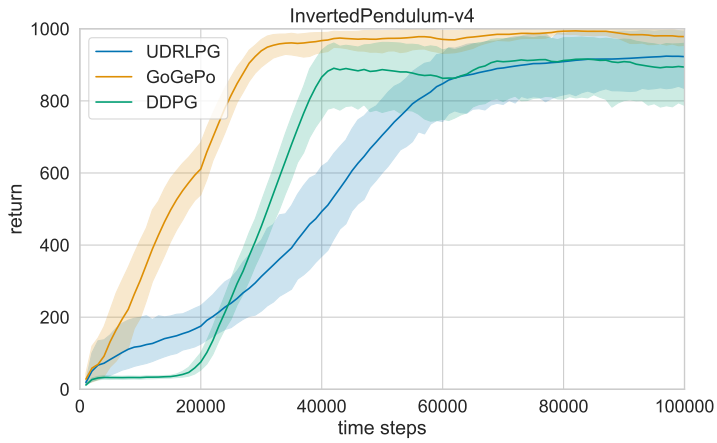


Figure 6.1. Performance of policies obtained with UDRLPG, GoGePo, and DDPG during training in InvertedPendulum-v4. Curves show the mean return and 95% bootstrapped confidence intervals from 20 independent runs as a function of environment interactions. All algorithms converge to comparable performance.

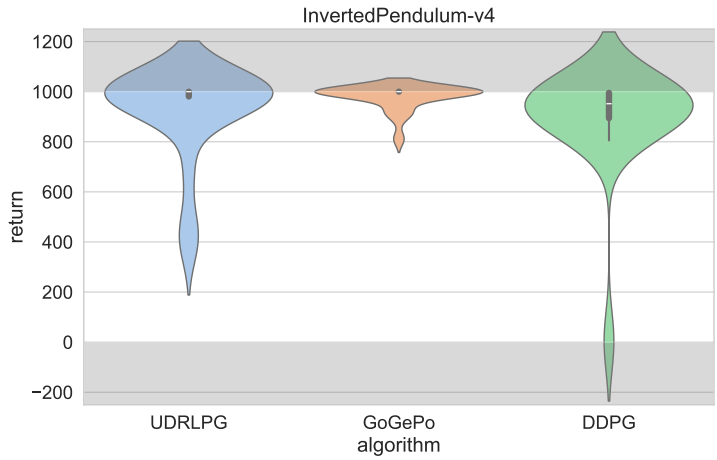


Figure 6.2. Variance of last returns obtained with UDRLPG, GoGePo, and DDPG in InvertedPendulum-v4: Mean return and 95% bootstrapped confidence intervals from 20 runs at final evaluation, as a function of environment interactions. GoGePo shows lower variance compared to the other methods.

Algorithm	Mean return	Std	Max	Min
UDRLPG	923	± 179	1000	390
GoGePo	978	± 48	1000	811
DDPG	894	± 212	1000	3

Table 6.1. Statistics for the last return (last evaluation) of 20 independent runs. Mean return is the mean of the last returns; std is the standard deviation of the last returns; max is the maximum last return; min is the minimum last reward (20 evaluations) for UDRLPG, GoGePo, and DDPG in InvertedPendulum-v4.

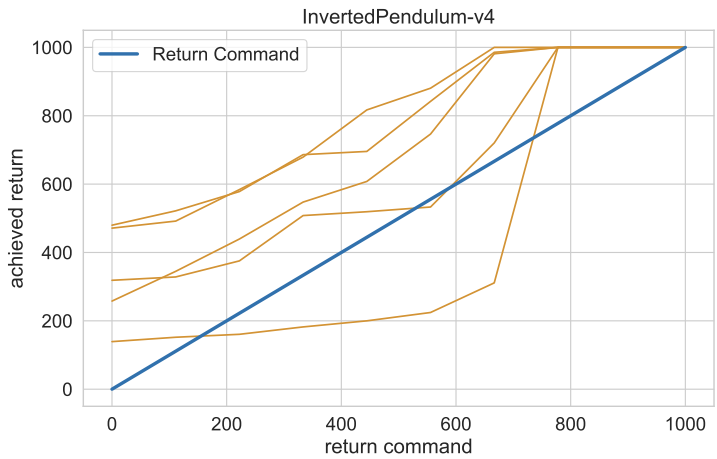


Figure 6.3. Mean returns over 10 episodes of policies from the generator as a function of the given command in InvertedPendulum-v4. Results are from five independent runs. The method can produce increasingly higher return policies.

6.2 Swimmer-v4

In this environment, UDRLPG demonstrates a distinct performance trajectory compared to the other algorithms. As illustrated in Figure 6.4, UDRLPG gradually improves its performance throughout the training process without reaching a plateau, achieving a mean return of approximately 300 by the end of the training period. This gradual improvement contrasts with GoGePo, which reaches a higher mean return of around 320 more rapidly, plateauing earlier in the training. DDPG exhibits limited progress, stabilizing around a lower mean return. Figure 6.5 provides further insights into the variance of the last returns. UDRLPG exhibits a broader spread, indicating more variability in the final returns across different runs, while GoGePo shows relatively low variance, with returns tightly clustered around its mean. UDRLPG achieves a higher maximum return than GoGePo and a lower minimum return, reflecting its higher variance. Table 6.2 summarizes the final returns obtained by each algorithm. Figure 6.6 demonstrates UDRLPG's capability of generating policies across the performance spectrum.

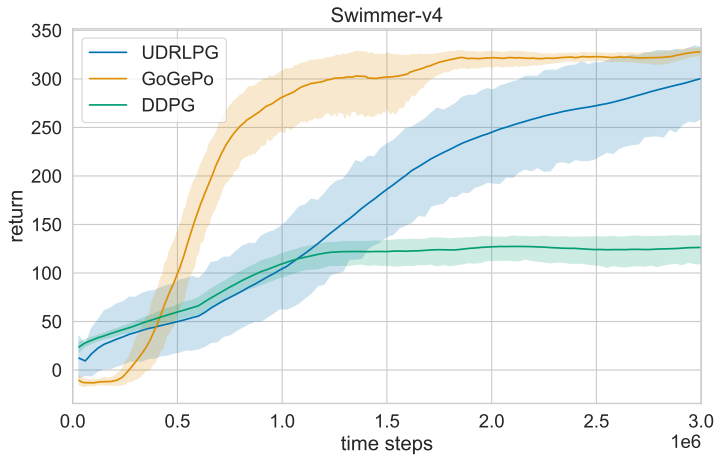


Figure 6.4. Performance of policies obtained with UDRLPG, GoGePo, and DDPG during training in Swimmer-v4. Curves show the mean return and 95% bootstrapped confidence intervals from 20 independent runs as a function of environment interactions. UDRLPG reaches a slightly lower return than GoGePo, while outperforming DDDPG.

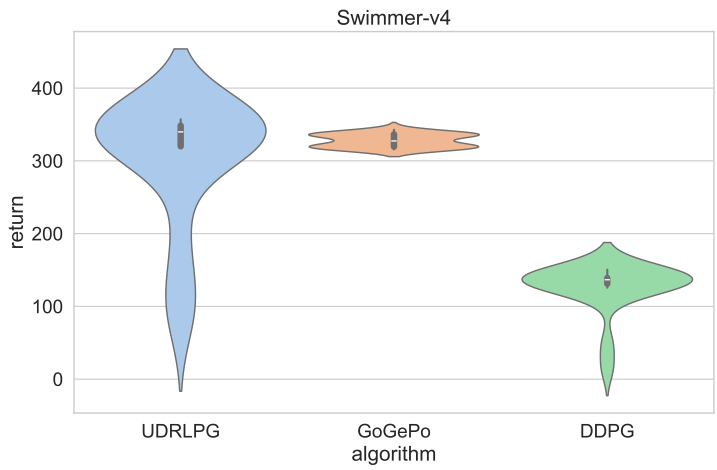


Figure 6.5. Variance of last returns obtained with UDRLPG, GoGePo, and DDPG in Swimmer-v4: Mean return and 95% bootstrapped confidence intervals from 20 runs at final evaluation, as a function of environment interactions. GoGePo shows lower variance compared to the other methods.

Algorithm	Mean return	Std	Max	Min
UDRLPG	300	± 86	**357**	80
GoGePo	328	± 9	343	316
DDPG	126	± 33	151	14

Table 6.2. Statistics for the last return (last evaluation) of 20 independent runs. Mean return is the mean of the last returns; std is the standard deviation of the last returns; max is the maximum last return; min is the minimum last reward (20 evaluations) for UDRLPG, GoGePo, and DDPG in Swimmer-v4.

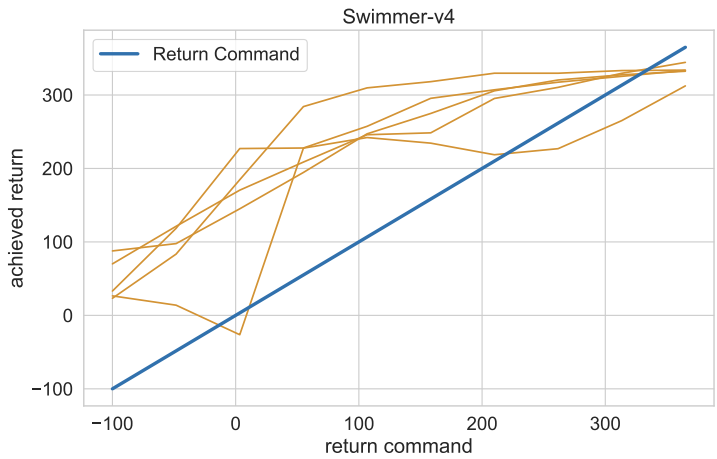


Figure 6.6. Mean returns over 10 episodes of policies from the generator as a function of the given command in Swimmer-v4. Results are from five independent runs. The method can produce increasingly higher return policies.

6.3 Hopper-v4

The performance of UDRLPG, as shown in Figure 6.7, demonstrates a convergence pattern that is comparable to the GoGePo algorithm, with both methods reaching a similar mean final return by the end of the training period in the Hopper-v4 environment. While UDRLPG exhibits a slightly slower learning curve initially compared to GoGePo, it ultimately achieves a comparable mean final return, as indicated in Table 6.3. UDRLPG shows substantial variability in performance, with a standard deviation of 908, which is again comparable to GoGePo's standard deviation of 1053. Figure 6.8 further illustrates the distribution of final returns. The data also reveals that the maximum return achieved by UDRLPG is notably higher, reaching a higher maximum compared to GoGePo. DDPG underperforms by a large margin compared to UDRLPG. In this environment, the proposed method is to be able to produce policies across the entire reward spectrum, as shown in Figure 6.9.

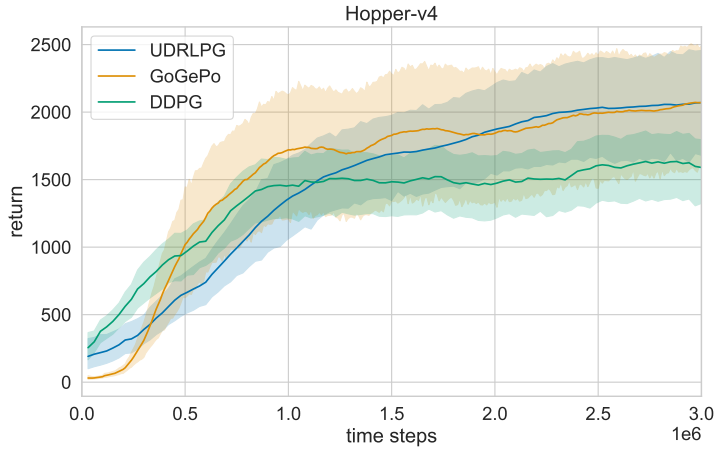


Figure 6.7. Performance of policies obtained with UDRLPG, GoGePo, and DDPG during training in Hopper-v4. Curves show the mean return and 95% bootstrapped confidence intervals from 20 independent runs as a function of environment interactions. UDRLPG and GoGePo converge to comparable performance.

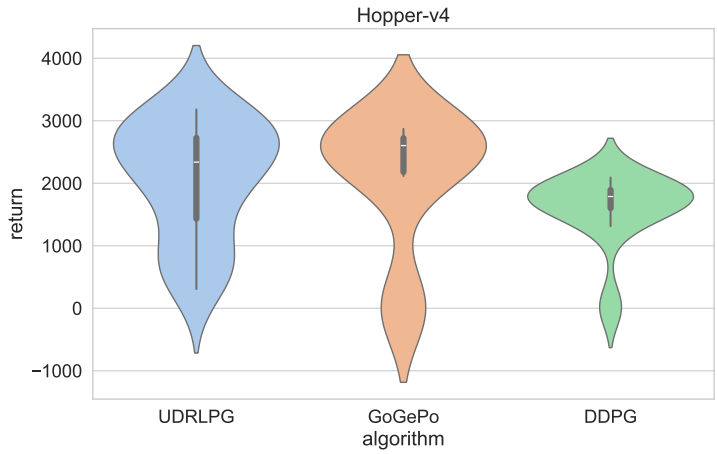


Figure 6.8. Variance of last returns obtained with UDRLPG, GoGePo, and DDPG in Hopper-v4: Mean return and 95% bootstrapped confidence intervals from 20 runs at final evaluation, as a function of environment interactions. Both GoGePo and UDRLPG show high variance. UDRLPG's last returns are distributed over the spectrum. GoGePo's last returns are concentrated into two clusters

Algorithm	Mean return	Std	Max	Min
UDRLPG	2070	± 908	**3180**	307
GoGePo	2071	± 1053	2869	3
DDPG	1590	± 558	2091	-1

Table 6.3. Statistics for the last return (last evaluation) of 20 independent runs. Mean return is the mean of the last returns; std is the standard deviation of the last returns; max is the maximum last return; min is the minimum last reward (20 evaluations) for UDRLPG, GoGePo, and DDPG in Hopper-v4.

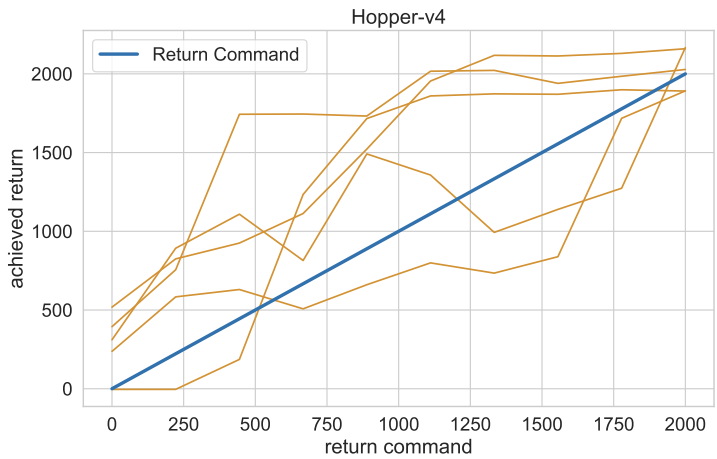


Figure 6.9. Mean returns over 10 episodes of policies from the generator as a function of the given command in Hopper-v4. Results are from five independent runs. The method can produce increasingly higher return policies.

Chapter 7

Discussion

The findings of this study provide several meaningful insights into the performance of the proposed method, UDRLPG. Compared to similar methods, particularly GoGePo, of which UDRLPG is a simplification, it demonstrates competitive performance while employing a significantly simpler architecture. Across all three environments used to assess the proposed algorithm, UDRLPG achieved a mean final return comparable to that of GoGePo. Additionally, UDRLPG outperformed DDPG in every environment except for InvertedPendulum, where it achieved similar performance.

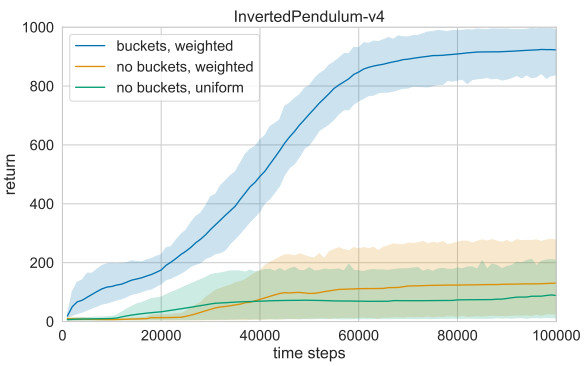
The study reveals interesting insights when analyzing the variance of the final returns achieved by UDRLPG and GoGePo across the environments. In both InvertedPendulum and Swimmer, UDRLPG exhibited higher variance compared to GoGePo. Interestingly, this is not the case in Hopper, where GoGePo showed a slightly higher variance than UDRLPG. The higher variance observed in GoGePo is due to learning stagnation in specific runs. Runs that did not suffer from stagnation converge consistently to final returns tightly clustered between 2400 and 2800. To ensure that the runs suffering from stagnation were not anomalies, the experiment with twenty independent runs was repeated multiple times, and the pattern of stagnation recurred consistently in each instance. Going back to UDRLPG, the high variance exhibited in Hopper and other environments cannot be attributed to a few stagnating runs—see Figure 6.8. Instead, it reflects a higher level of variability in the learning process. This increased variability in the learning process may also explain UDRLPG's ability to achieve consistently higher maximum final returns compared to GoGePo, a trend present in both Swimmer and Hopper. In Swimmer, although the difference in maximum return between UDRLPG and GoGePo is relatively small, it remains noteworthy, especially because that UDRLPG's learning curve shows no signs of plateauing, indicating possible room for further improvement. GoGePo instead appears to stabilize around 2 million timesteps. In Hopper, the difference is substantial, with UDRLPG reaching a maximum last return of 3180, around 300 higher than the maximum achieved by GoGePo. This result suggests that, under the right conditions, UDRLPG can explore the search space more extensively, leading to higher returns.

UDRLPG shows promising results when analyzing its ability to produce policies across the entire reward spectrum. InvertedPendulum is the most challenging environment to obtain an optimal identity curve. Unintuitively, this complexity derives from the fact that InvertedPendulum is a simple environment where a single perturbation of a low-performing policy can turn it into an optimal policy, that is, making the pendulum stay upright for all the 1000 timesteps and achieve a maximum reward. This explains why some of the curves obtained resemble a step function, jumping from low to high rewards with little to no intermediate values. This phenomenon can occur when no examples of policies achieving rewards in other parts of the spectrum exist in the replay buffer. For this reason, the generator can struggle to learn to produce policies that achieve such rewards. Despite this challenge, UDRLPG can produce strong identity curves in most cases, as shown in Figure 6.3, suggesting robust generalization over commands. Producing the desired identity is less complex in the other two environments because, during training, examples of policies across the entire spectrum will be generated almost certainly, as perturbing a policy will most likely not make it achieve a notably higher reward than before, making it more likely for the buffer to be populated with policies across every intermediate return. This, in turn, makes it easier for the decoder to learn to produce intermediate returns.

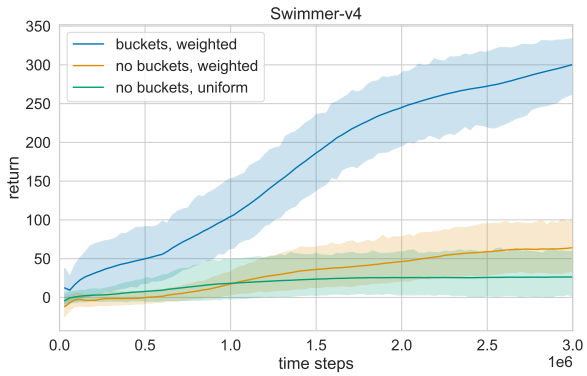
During the development of this method, the addition of performance buckets to force a more even distribution within the buffer proved to be a critical addition. Therefore, in Figure 7.1, we provide a

comparison between three strategies applied to the replay buffer: no performance buckets and uniform sampling, no performance buckets and weighted sampling, and lastly, the method proposed in this study using performance buckets along with weighted sampling. When using the buffer as-is, without performance buckets, even when applying a weighting strategy, the difference in performance is staggering. It must be noted, however, that this experiment was run using the same hyperparameters presented in Table 5.1, which are the result of extensive tuning with a replay buffer and performance buckets. Tuning the hyperparameters once again without buckets could lead to better performance.

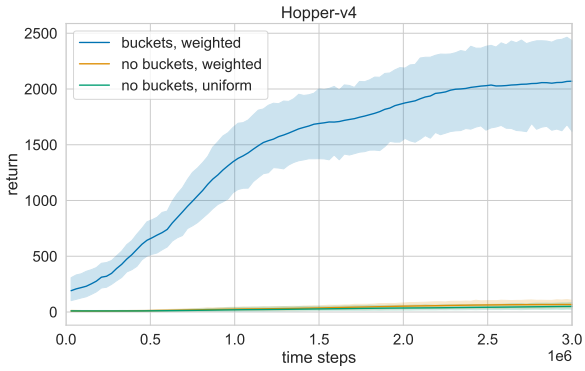
While buckets drastically reduce the likelihood of stagnation during training. Convergence speed can be greatly improved by biasing the learning process towards higher-performing policies. Figure 7.2 demonstrates the effectiveness of the proposed weighting strategy utilized to sample from the buffer. Convergence speed improves across all environments. InvetedPendulum and Swimmer are the environments where this strategy allows for the most notable performance improvement.



(a) InvertedPendulum-v4, performance buckets vs. no performance buckets

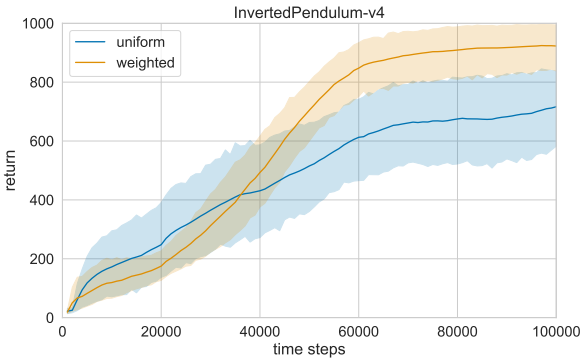


(b) Swimmer-v4, performance buckets vs. no performance buckets

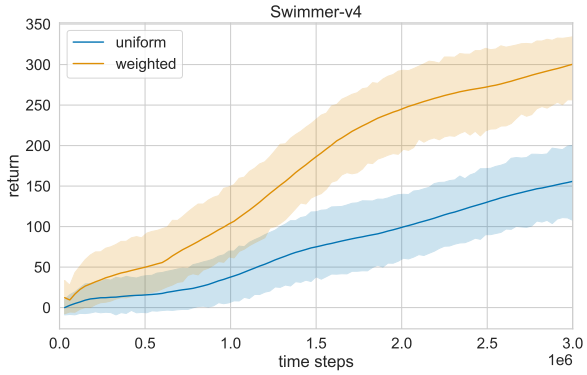


(c) Hopper-v4, performance buckets vs. no performance buckets

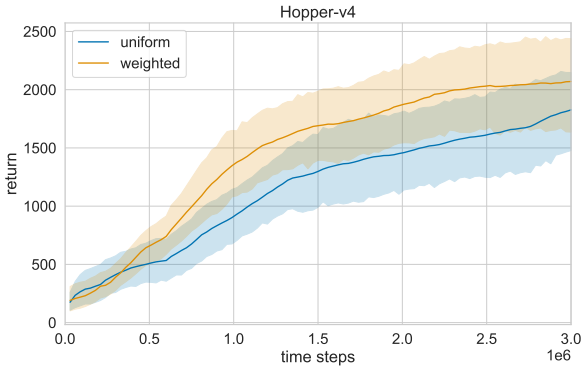
Figure 7.1. Performance of policies obtained with UDRLPG during training in InvertedPendulum-v4 (a), Swimmer-v4 (b), and Hopper-v4 (c), using three different strategies for the replay buffer: performance buckets with weighted sampling (the proposed approach); no performance buckets with uniform sampling; no performance buckets with weighted sampling. Curves show the mean return and 95% bootstrapped confidence intervals from 20 independent runs as a function of environment interactions. Weighting the buffer without buckets outperforms a uniform strategy. The proposed strategy outperforms all other strategies in every environment.



(a) InvertedPendulum-v4, weighted vs. uniform



(b) Swimmer-v4, weighted vs. uniform



(c) Hopper-v4, weighted vs. uniform

Figure 7.2. Performance of policies obtained with UDRLPG during training in InvertedPendulum-v4 (a), Swimmer-v4 (b), and Hopper-v4 (c), using either uniform or weighted sampling from the replay buffer. Curves show the mean return and 95% bootstrapped confidence intervals from 20 independent runs as a function of environment interactions. The proposed weighting strategy outperforms uniform weighting in all environments.

As previously mentioned in Chapters 3 and 4, UDRLPG inherits from UDRL potential challenges deriving from the multimodality issue. In the proposed method, this problem presents itself as such: the newly generated policy θ_{new}, obtained by conditioning the generator on a command c may be significantly different from the policies θ in the buffer, where $R(\theta) = c$. If this happens, the generator will try to fit the mean of the two policies θ_{new} and θ, say θ_{mean}. However, there is no guarantee that $R(\theta_{mean}) \approx R(\theta_{new}) = R(\theta)$, that is, the expected reward of θ_{mean} is not guaranteed to have similar expected return to the policies from which it originates. In this case, the learning process may start degrading. Fortunately, this issue does not arise frequently. One possible explanation is that, early on during training, the hypernetwork may develop an understanding of the underlying structure or distribution of the policy weights based on the first policies in the buffer. Figure 7.4 shows t-SNE (Van der Maaten and Hinton [2008]) (t-distributed Stochastic Neighbor Embedding)—a dimensionality reduction algorithm—applied to all the policies contained in the buffers of five independent runs. The tight, non-overlapping clusters further support the intuition that weight initialization biases the hypernetwork towards a specific arrangement of the neural network's weights (Chen et al. [1993]). This arrangement includes the order of the neurons within hidden layers (neuron permutation) and the magnitudes of the weights (weight scaling). The bias induced by the initialization constrains the search space in a part of the solution space where all weights follow the same configuration. The hypernetwork learns changes around the configuration. This reduces the likelihood of generating policies with similar returns that are very different in weight space. This would, in turn, degrade learning due to the hypernetwork learning to fit the mean of two policies with the same return but different weight configurations, which does not necessarily have the same return.

The configuration learned by the hypernetwork depends on the weight initialization. If this were not the case, we would expect to see policies generated at the beginning of training lie out of the clusters because they would not follow the configuration. Instead, we observe tight clusters, suggesting that the hypernetwork is biased by the weight initialization itself.

This learned structure biases the hypernetwork towards generating new policies similar to those in the buffer, reducing the chances of generating drastically different policies that could cause instability. To further explore this issue, during the development of this method, we attempted to train the same model used in UDRLPG with an offline dataset created by combining several buffers dumped from multiple independent runs. Following the intuition presented before, each run's generator could have learned a different distribution over the weights of the policies. In this case, the dataset obtained by combining policies created by the generators during independent runs may contain vastly different policies that a single hypernetwork will struggle to learn. Our experiments showed that it was indeed significantly more difficult for the generator to learn to produce meaningful policies.

Figure 7.3 shows the result of applying t-SNE to the policies in a buffer at the end of a run. This plot reveals common patterns in the policy landscape shared among all the runs: a dense yellow area in the middle where high-return policies are concentrated and an elongated purple region representing lower-return policies. A performance gradient is visible in all buckets, transitioning from deep purple (low performance) to yellow (high performance). This indicates that moving through the space in specific directions correlates with improvements or declines in policy performance.

If not tuned correctly, the perturbations applied to the generated policies could become another source of instability. If the scale of the Gaussian noise added to the policies is too great, it may push the policy outside the distribution learned by the hypernetwork, leading to the aforementioned issue. Proper tuning of this hyperparameter is essential for this method, as the noise must be sufficient to explore the search space while not causing instability.

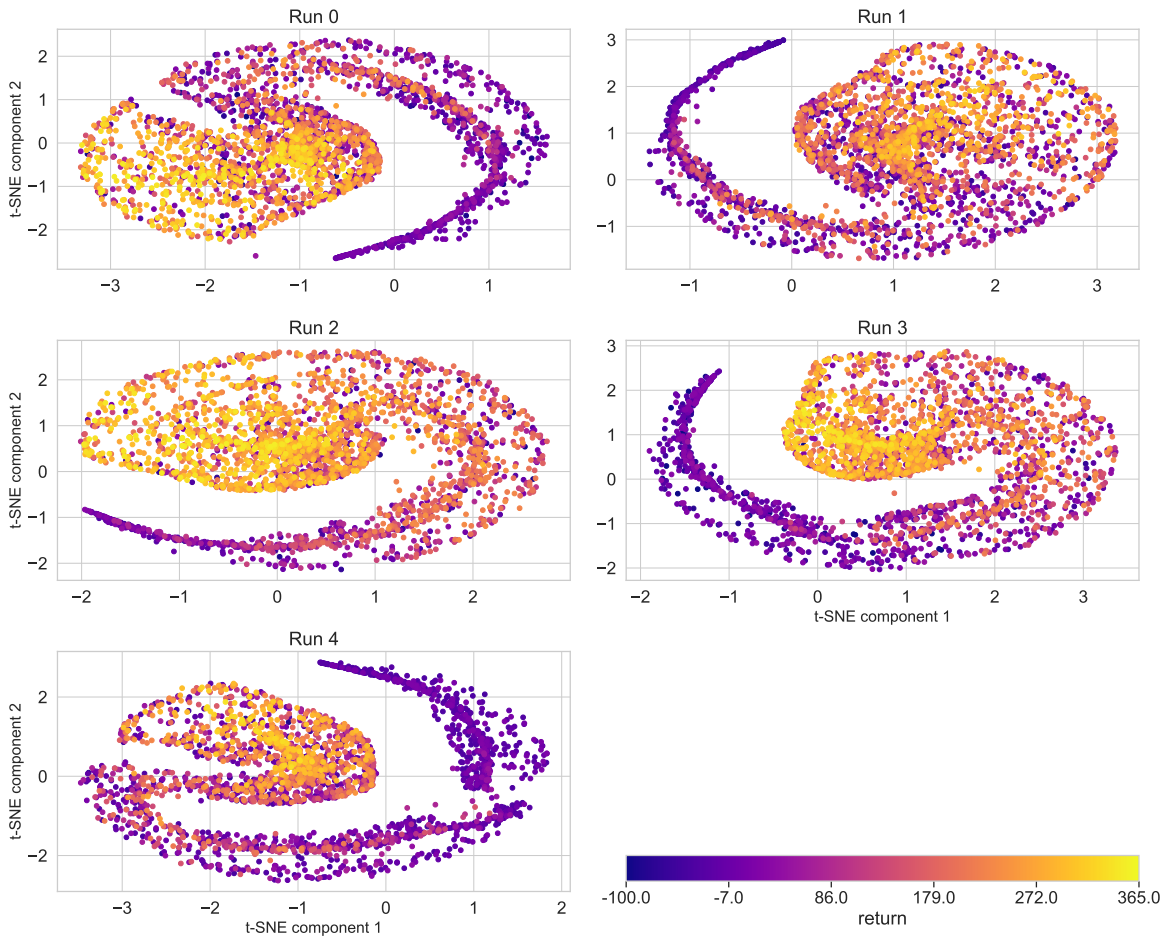


Figure 7.3. t-SNE applied indepentently to each buffer of five independent runs. Color gradient represents the return of each policy. Buffer dumped at the end of training. Each buffer contains 2999 policies. Environment: Swimmer-v4

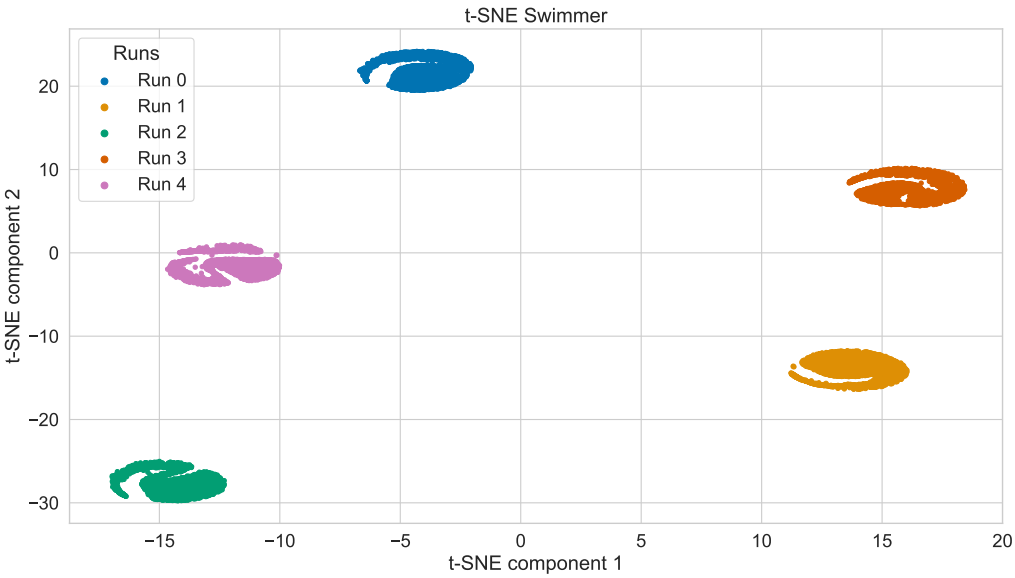


Figure 7.4. t-SNE applied to policies in the buffers of five independent runs. Buffers dumped at the end of training and merged together. Environment: Swimmer-v4

Chapter 8

Conclusion

This thesis introduced Upside Down Reinforcement Learning with Policy Generators (UDRLPG), an approach to reinforcement learning focused on generating policies conditioned on return commands. The primary objective was to address the architectural complexity of similar methods while retaining the advantages obtained in terms of sample complexity. UDRLPG removes the need for a separate evaluator in the architecture, thus simplifying the overall structure.

The results obtained from the evaluation of UDRLPG against two baseline algorithms, GoGePo and DDPG, demonstrated its ability to converge to competitive mean returns when compared to GoGePo, whilst outperforming DDPG. Its capacity to explore the parameter space more extensively, resulting in higher return upper bounds, is also a notable result. The proposed method also showed a remarkable ability to consistently produce policies across the reward spectrum in all environments, demonstrating generalization abilities over commands.

The analysis of the policies generated during training provides interesting insights into how the bias induced by the initialization of the hypernetwork is central to this method's success. Our findings suggest that this bias effectively constrains the search in a part of the solution space where all weights follow the same configuration. As a byproduct of this phenomenon, the hypernetwork avoids the issue of multimodality.

UDRLPG also exhibited some limitations. In some environments, its learning process displayed slower convergence and significant variability, leading to higher variance in final returns across runs compared to GoGePo. Despite these challenges, by applying other techniques to stabilize training, UDRLPG can learn to produce high-reward policies in various environments without the need for an evaluator. Compared to GoGePo, UDRLPG simplifies the learning process and opens new possibilities for further exploration into goal-conditioned policy generation.

8.1 Future Work

In the future, it would be beneficial to concentrate on improving upon UDRLPG's high variance by using other techniques to stabilize training further. Integrating more robust mechanisms to address the multimodality problem may prove beneficial in enhancing the capabilities of this method. Future research could also focus on increasing UDRLPG's stability and consistency by integrating more robust mechanisms to lower variance during training. Extending the approach to more complex, high-dimensional environments could further validate UDRLPG's versatility and adaptability.

Our attempts to use Variational Autoencoders to address this problem have not yielded definitive results. However, this does not preclude the potential for further refinement and development of this approach. As such, there remains significant scope for continued investigation into this method, which may lead to more effective solutions.

Glossary

Bibliography

Dana H Ballard. Modular learning in neural networks. In *Proceedings of the sixth National conference on Artificial intelligence-Volume 1*, pages 279–284, 1987.

Christopher Berner, Greg Brockman, Brooke Chan, Vicki Cheung, Przemysław Dębiak, Christy Dennison, David Farhi, Quirin Fischer, Shariq Hashme, Chris Hesse, et al. Dota 2 with large scale deep reinforcement learning. *arXiv preprint arXiv:1912.06680*, 2019.

Samuel R Bowman, Luke Vilnis, Oriol Vinyals, Andrew M Dai, Rafal Jozefowicz, and Samy Bengio. Generating sentences from a continuous space. *arXiv preprint arXiv:1511.06349*, 2015.

Bruce G Buchanan. A (very) brief history of artificial intelligence. *Ai Magazine*, 26(4):53–53, 2005.

An Mei Chen, Haw-minn Lu, and Robert Hecht-Nielsen. On the geometry of feedforward neural network error surfaces. *Neural computation*, 5(6):910–927, 1993.

Lili Chen, Kevin Lu, Aravind Rajeswaran, Kimin Lee, Aditya Grover, Misha Laskin, Pieter Abbeel, Aravind Srinivas, and Igor Mordatch. Decision transformer: Reinforcement learning via sequence modeling. *Advances in neural information processing systems*, 34:15084–15097, 2021.

Yan Duan, Xi Chen, Rein Houthooft, John Schulman, and Pieter Abbeel. Benchmarking deep reinforcement learning for continuous control. *CoRR*, abs/1604.06778, 2016. URL `http://arxiv.org/abs/1604.06778`.

Francesco Faccio, Louis Kirsch, and Jürgen Schmidhuber. Parameter-based value functions. In *International Conference on Learning Representations*, 2021. URL `https://openreview.net/forum?id=tV6oBfuyLTQ`.

Francesco Faccio, Vincent Herrmann, Aditya Ramesh, Louis Kirsch, and Jürgen Schmidhuber. Goal-conditioned generators of deep policies. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 37, pages 7503–7511, 2023.

Kunihiko Fukushima. Visual feature extraction by a multilayered network of analog threshold elements. *IEEE Transactions on Systems Science and Cybernetics*, 5(4):322–333, 1969. doi: 10.1109/TSSC.1969.300225.

Faustino J Gomez and Jürgen Schmidhuber. Co-evolving recurrent neurons learn deep memory pomdps. In *Proceedings of the 7th annual conference on Genetic and evolutionary computation*, pages 491–498, 2005.

Noah J Goodall. Machine ethics and automated vehicles. *Road vehicle automation*, pages 93–102, 2014.

Karol Gregor. Finding online neural update rules by learning to remember. *arXiv preprint arXiv:2003.03124*, 2020.

David Ha, Andrew Dai, and Quoc V Le. Hypernetworks. *arXiv preprint arXiv:1609.09106*, 2016.

Jean Harb, Tom Schaul, Doina Precup, and Pierre-Luc Bacon. Policy evaluation networks. *arXiv preprint arXiv:2002.11833*, 2020.

Florent Hiard. EPFL and DeepMind use AI to control plasmas for nuclear fusion — actu.epfl.ch. https://actu.epfl.ch/news/epfl-and-deepmind-use-ai-to-control-plasmas-for-nu/, 2022. [Accessed 27-08-2024].

S Hochreiter. Long short-term memory. *Neural Computation MIT-Press*, 1997.

Kazuki Irie, Imanol Schlag, Róbert Csordás, and Jürgen Schmidhuber. A modern self-referential weight matrix that learns to modify itself. In *International Conference on Machine Learning*, pages 9660–9677. PMLR, 2022.

Michael Janner, Qiyang Li, and Sergey Levine. Offline reinforcement learning as one big sequence modeling problem. *Advances in neural information processing systems*, 34:1273–1286, 2021.

John Jumper, Richard Evans, Alexander Pritzel, Tim Green, Michael Figurnov, Olaf Ronneberger, Kathryn Tunyasuvunakool, Russ Bates, Augustin Žídek, Anna Potapenko, et al. Highly accurate protein structure prediction with alphafold. *nature*, 596(7873):583–589, 2021.

Leslie Pack Kaelbling, Michael L. Littman, and Andrew W. Moore. Reinforcement learning: A survey. *CoRR*, cs.AI/9605103, 1996. URL https://arxiv.org/abs/cs/9605103.

Sham Machandranath Kakade. *On the sample complexity of reinforcement learning*. University of London, University College London (United Kingdom), 2003.

Diederik P Kingma. Auto-encoding variational bayes. *arXiv preprint arXiv:1312.6114*, 2013.

Louis Kirsch and Jürgen Schmidhuber. Meta learning backpropagation and improving it. *Advances in Neural Information Processing Systems*, 34:14122–14134, 2021.

Louis Kirsch, Sebastian Flennerhag, Hado Van Hasselt, Abram Friesen, Junhyuk Oh, and Yutian Chen. Introducing symmetries to black box meta reinforcement learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 36, pages 7202–7210, 2022.

Boris Knyazev, Michal Drozdzal, Graham W. Taylor, and Adriana Romero-Soriano. Parameter prediction for unseen deep architectures. *CoRR*, abs/2110.13100, 2021. URL https://arxiv.org/abs/2110.13100.

Věra Kůrková and Paul C Kainen. Functionally equivalent feedforward neural networks. *Neural Computation*, 6(3):543–558, 1994.

Gottfried Wilhelm Leibniz. *The early mathematical manuscripts of Leibniz*. Courier Corporation, 2012.

TP Lillicrap. Continuous control with deep reinforcement learning. *arXiv preprint arXiv:1509.02971*, 2015.

Seppo Linnainmaa. *The representation of the cumulative rounding error of an algorithm as a Taylor expansion of the local rounding errors*. PhD thesis, Master's Thesis (in Finnish), Univ. Helsinki, 1970.

Horia Mania, Aurelia Guy, and Benjamin Recht. Simple random search of static linear policies is competitive for reinforcement learning. *Advances in neural information processing systems*, 31, 2018.

John McCarthy, Marvin L Minsky, Nathaniel Rochester, and Claude E Shannon. A proposal for the dartmouth summer research project on artificial intelligence, august 31, 1955. *AI magazine*, 27(4): 12–12, 2006.

Pamela McCorduck and Cli Cfe. *Machines who think: A personal inquiry into the history and prospects of artificial intelligence*. AK Peters/CRC Press, 2004.

Thomas Miconi, Kenneth Stanley, and Jeff Clune. Differentiable plasticity: training plastic neural networks with backpropagation. In *International Conference on Machine Learning*, pages 3559–3568. PMLR, 2018.

Volodymyr Mnih. Playing atari with deep reinforcement learning. *arXiv preprint arXiv:1312.5602*, 2013.

Elias Najarro and Sebastian Risi. Meta-learning through hebbian plasticity in random networks. *Advances in Neural Information Processing Systems*, 33:20719–20731, 2020.

Martin L Puterman. *Markov decision processes: discrete stochastic dynamic programming*. John Wiley & Sons, 2014.

R Rajarshi. Designing Arithmetic Circuits with Deep Reinforcement Learning | NVIDIA Technical Blog — developer.nvidia.com. `https://developer.nvidia.com/blog/designing-arithmetic-circuits-with-deep-reinforcement-learning/`, 2022. [Accessed 27-08-2024].

Sahand Rezaei-Shoshtari, Charlotte Morissette, Francois Robert Hogan, Gregory Dudek, and David Meger. Hypernetworks for zero-shot transfer in reinforcement learning, 2023. URL `https://arxiv.org/abs/2211.15457`.

Tim Salimans, Jonathan Ho, Xi Chen, Szymon Sidor, and Ilya Sutskever. Evolution strategies as a scalable alternative to reinforcement learning. *arXiv preprint arXiv:1703.03864*, 2017a.

Tim Salimans, Jonathan Ho, Xi Chen, Szymon Sidor, and Ilya Sutskever. Evolution strategies as a scalable alternative to reinforcement learning, 2017b. URL `https://arxiv.org/abs/1703.03864`.

Imanol Schlag, Kazuki Irie, and Jürgen Schmidhuber. Linear transformers are secretly fast weight programmers. In *International Conference on Machine Learning*, pages 9355–9366. PMLR, 2021.

Juergen Schmidhuber. Reinforcement learning upside down: Don't predict rewards–just map them to actions. *arXiv preprint arXiv:1912.02875*, 2019.

Jürgen Schmidhuber. Learning to generate sub-goals for action sequences. In *Artificial neural networks*, pages 967–972, 1991.

Jürgen Schmidhuber. Learning to control fast-weight memories: An alternative to dynamic recurrent networks. *Neural Computation*, 4(1):131–139, 1992.

Jürgen Schmidhuber. A 'self-referential'weight matrix. In *ICANN'93: Proceedings of the International Conference on Artificial Neural Networks Amsterdam, The Netherlands 13–16 September 1993 3*, pages 446–450. Springer, 1993.

Jürgen Schmidhuber. On learning to think: Algorithmic information theory for novel combinations of reinforcement learning controllers and recurrent neural world models. *arXiv preprint arXiv:1511.09249*, 2015.

Jürgen Schmidhuber and Reiner Wahnsiedler. Planning simple trajectories using neural subgoal generators. 1993.

Frank Sehnke, Christian Osendorfer, Thomas Rückstieß, Alex Graves, Jan Peters, and Jürgen Schmidhuber. Policy gradients with parameter-based exploration for control. In *Artificial Neural Networks-ICANN 2008: 18th International Conference, Prague, Czech Republic, September 3-6, 2008, Proceedings, Part I 18*, pages 387–396. Springer, 2008a.

Frank Sehnke, Christian Osendorfer, Thomas Rückstieß, Alex Graves, Jan Peters, and Jürgen Schmidhuber. Policy gradients with parameter-based exploration for control. In Véra Kůrková, Roman Neruda, and Jan Koutník, editors, *Artificial Neural Networks - ICANN 2008*, pages 387–396, Berlin, Heidelberg, 2008b. Springer Berlin Heidelberg. ISBN 978-3-540-87536-9.

Frank Sehnke, Christian Osendorfer, Thomas Rückstieß, Alex Graves, Jan Peters, and Jürgen Schmidhuber. Parameter-exploring policy gradients. *Neural Networks*, 23(4):551–559, 2010a.

Frank Sehnke, Christian Osendorfer, Thomas Rückstieß, Alex Graves, Jan Peters, and Jürgen Schmidhuber. Parameter-exploring policy gradients. *Neural Networks*, 23(4):551–559, May 2010b. ISSN 08936080. doi: 10.1016/j.neunet.2009.12.004.

Noel Sharkey. A 13th century programmable robot. *Univ. of Sheffield*, 2007a.

Noel Sharkey. A programmable robot from 60 ad, 2611. *New Scientist*, 2007b.

David Silver, Julian Schrittwieser, Karen Simonyan, Ioannis Antonoglou, Aja Huang, Arthur Guez, Thomas Hubert, Lucas Baker, Matthew Lai, Adrian Bolton, et al. Mastering the game of go without human knowledge. *nature*, 550(7676):354–359, 2017.

Kihyuk Sohn, Honglak Lee, and Xinchen Yan. Learning structured output representation using deep conditional generative models. *Advances in neural information processing systems*, 28, 2015.

Rupesh Kumar Srivastava, Pranav Shyam, Filipe Mutz, Wojciech Jaśkowski, and Jürgen Schmidhuber. Training agents using upside-down reinforcement learning. *arXiv preprint arXiv:1912.02877*, 2019.

Stephen M Stigler. Gauss and the invention of least squares. *the Annals of Statistics*, pages 465–474, 1981.

RL Stratonovich. Conditional Markov processes. *Theory of Probability And Its Applications*, 5(2):156–178, 1960.

Miroslav Štrupl, Francesco Faccio, Dylan R Ashley, Jürgen Schmidhuber, and Rupesh Kumar Srivastava. Upside-down reinforcement learning can diverge in stochastic environments with episodic resets. *arXiv preprint arXiv:2205.06595*, 2022.

Richard S Sutton. Generalization in reinforcement learning: Successful examples using sparse coarse coding. *Advances in neural information processing systems*, 8, 1995.

Richard S Sutton. Reinforcement learning: An introduction. *A Bradford Book*, 2018.

Richard S Sutton, David McAllester, Satinder Singh, and Yishay Mansour. Policy gradient methods for reinforcement learning with function approximation. *Advances in neural information processing systems*, 12, 1999.

Richard Stuart Sutton. *Temporal credit assignment in reinforcement learning.* University of Massachusetts Amherst, 1984.

Gerald Tesauro. Td-gammon, a self-teaching backgammon program, achieves master-level play. *Neural computation*, 6(2):215–219, 1994.

Mark Towers, Ariel Kwiatkowski, Jordan Terry, John U Balis, Gianluca De Cola, Tristan Deleu, Manuel Goulão, Andreas Kallinteris, Markus Krimmel, Arjun KG, et al. Gymnasium: A standard interface for reinforcement learning environments. *arXiv preprint arXiv:2407.17032*, 2024.

Laurens Van der Maaten and Geoffrey Hinton. Visualizing data using t-sne. *Journal of machine learning research*, 9(11), 2008.

Christopher JCH Watkins and Peter Dayan. Q-learning. *Machine learning*, 8:279–292, 1992.

Daan Wierstra, Alexander Foerster, Jan Peters, and Juergen Schmidhuber. Solving deep memory pomdps with recurrent policy gradients. In *Artificial Neural Networks–ICANN 2007: 17th International Conference, Porto, Portugal, September 9-13, 2007, Proceedings, Part I 17*, pages 697–706. Springer, 2007.

Ronald J Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine learning*, 8:229–256, 1992.

Qinqing Zheng, Amy Zhang, and Aditya Grover. Online decision transformer, 2022. URL `https://arxiv.org/abs/2202.05607`.